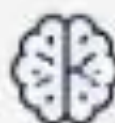


XÂY DỰNG HỆ THỐNG AI TÁC NHÂN ĐÁNG TIN CẬY

*Bài học kỹ thuật từ một hệ thống Agentic RAG
trong ngành dược phẩm*



Một cuốn sách
tổng hợp và diễn giải lại



Biên soạn với sự hỗ trợ
của Claude (Anthropic)



Tháng 8, 2026

Mục Lục

Lời nói đầu.....	5
Chương 1 Vì sao ngành dược phẩm cần một trợ lý AI biết đọc PDF	6
Một ngày làm việc điển hình, trước và sau	7
Vì sao tìm kiếm từ khoá truyền thống không đủ.....	7
Chương 2 Hành trình ba giai đoạn: Tìm kiếm, Hỏi đáp, rồi Hành động.....	9
Giai đoạn 1: Tìm kiếm (Search).....	9
Giai đoạn 2: Hỏi đáp (Ask)	9
Giai đoạn 3: Hành động (Do).....	9
Vì sao thứ tự này quan trọng	9
Chương 3 Bức tranh kiến trúc tổng thể	11
Vì sao chọn kiến trúc đa tác nhân thay vì một mô hình duy nhất.....	13
Vai trò của LangGraph như một "nhạc trưởng".....	13
Chương 4 Hai trụ cột: Kỹ thuật ngữ cảnh và Kỹ thuật giàn giáo.....	14
Kỹ thuật ngữ cảnh là gì?	14
Kỹ thuật giàn giáo là gì?	14
Một phép ẩn dụ dễ hình dung.....	16
Vì sao hai khái niệm này thường bị bỏ qua.....	16
Chương 5 Bước đầu tiên: Làm rõ ý định người dùng	17
Cân bằng giữa hỏi lại và làm phiền người dùng	17
Bài học cho người xây dựng hệ thống của riêng bạn.....	18
Chương 6 Suy nghĩ trước khi hành động: bước Think & Plan	19
Vì sao bước này trở nên thiết yếu khi hệ thống có nhiều công cụ	19
Vì sao "không gian để suy nghĩ" lại hiệu quả đến vậy	19
Chương 7 Agent Nghiên cứu: khi RAG bắt tay với SQL	21
Hai chiến lược truy xuất song song.....	22
Bên trong pipeline RAG: một dây chuyền bảy bước	23
Text-to-SQL: khi câu hỏi cần con số chính xác, không phải đoạn văn	24
Vì sao phải mở rộng câu hỏi thành năm biến thể.....	25
Vì sao cần bước xếp hạng lại (rerank) sau khi đã tìm kiếm lại.....	25
Chương 8 Agent Phản tư: dữ liệu đã đủ để trả lời chưa?	27
Vì sao "biết mình chưa biết đủ" lại khó với AI.....	27
Follow-up questions: câu nối giữa phản tư và hành động tiếp theo	27
Chương 9 Agent Viết: từ bằng chứng thô đến câu trả lời có trách nhiệm	29

Ba vòng phản tư bổ sung cho nhau	29
Vì sao trích dẫn không phải là một tính năng "gắn thêm"	31
Chương 10 Niềm tin bắt đầu từ sự minh bạch.....	32
Hiển thị các bước trung gian	32
Trích dẫn chi tiết đến từng câu.....	32
Sự khác biệt giữa "nghe có vẻ đúng" và "kiểm chứng được"	32
Hiệu ứng phụ: trích dẫn tốt giúp rút ngắn thời gian rà soát	33
Chương 11 Đánh giá và giám sát: đo lường điều không thể thấy bằng mắt thường... 34	
Đánh giá theo bộ dữ liệu chuẩn (Dataset Evaluations)	34
Đánh giá trên lưu lượng thực tế (Live Traffic Evaluations)	34
Giám sát liên tục.....	34
"Kim tự tháp kiểm thử" áp dụng cho hệ thống agentic	34
Vì sao cần cả hai loại đánh giá, không chỉ một.....	35
Chương 12 Khi mọi thứ đi sai: xử lý lỗi và phục hồi trạng thái	37
Lưu trạng thái bền vững.....	37
Thử lại tự động và thử lại theo yêu cầu người dùng	37
Dự phòng mô hình (LLM Fallbacks)	37
Chi phí và lợi ích của việc lưu trạng thái sau mỗi bước	38
Khi nào nên tự động thử lại, và khi nào nên dừng lại chờ con người.....	38
Chương 13 Dữ liệu sạch hơn, hệ thống thông minh hơn	40
Vòng lặp tự củng cố giữa chất lượng dữ liệu và chất lượng câu trả lời.....	40
Chương 14 Phát triển lặp: đừng chờ hoàn hảo mới ra mắt.....	42
Một thứ tự ưu tiên đáng suy ngẫm: chất lượng trước, chi phí sau	42
Chương 15 Kết luận: bài học cho những ai đang xây hệ thống agentic AI của riêng mình	43
Chương 16 Ví dụ thực tế 2026: agentic AI ngoài ngành dược	45
Tài chính: JPMorgan Chase và quy mô vận hành công nghiệp.....	45
Tài chính: Morgan Stanley và bài học về phạm vi sử dụng.....	45
Bán lẻ và dịch vụ khách hàng: bài học đảo chiều của Klarna.....	45
Pháp lý: Harvey AI và cái giá của việc thiếu kỷ luật trích dẫn.....	46
Sản xuất và chuỗi cung ứng: RAG trên tài liệu kỹ thuật	46
Điểm chung xuyên suốt các ví dụ 2026	47
Chương 17 Checklist thực hành cho đội ngũ của bạn	48
Trước khi bắt đầu: đánh giá nền tảng dữ liệu	48
Khi thiết kế kiến trúc	48

Khi xây dựng độ tin cậy	48
Khi thiết kế khả năng phục hồi	49
Khi vận hành lâu dài	49
Chương 18 Câu hỏi thường gặp	50
Hệ thống này có thể áp dụng cho ngành khác ngoài dược phẩm không?	50
Có nhất thiết phải dùng LangGraph không?	50
Một đội ngũ nhỏ, ít tài nguyên, có thể áp dụng những bài học này không?	50
Làm sao biết khi nào hệ thống của tôi đã "đủ tin cậy" để đưa vào sản xuất?	50
AI có thay thế hoàn toàn vai trò của chuyên gia con người trong quy trình này không?	51
Phụ lục Bảng thuật ngữ	52
Nguồn tham khảo và ghi nhận	54
Lời cảm ơn từ người biên soạn	54

Lời nói đầu

Trong vài năm gần đây, cụm từ "AI Agent" (tác nhân AI) xuất hiện ở khắp nơi từ các bài demo ấn tượng trên mạng xã hội cho đến những lời hứa hẹn về việc "AI sẽ tự làm mọi việc thay con người". Nhưng có một khoảng cách rất lớn giữa một demo chạy mượt trên vài câu hỏi mẫu, và một hệ thống thực sự chạy ổn định hàng ngày, phục vụ hàng trăm nhà nghiên cứu, xử lý dữ liệu nhạy cảm, và phải chịu trách nhiệm trước các quy định ngặt nghèo của ngành dược phẩm.

Cuốn sách nhỏ này ra đời từ một câu hỏi đơn giản: một hệ thống agentic AI thực sự đáng tin cậy trong môi trường doanh nghiệp trông như thế nào, và người kỹ sư cần nghĩ về điều gì để xây dựng được nó? Để trả lời, chúng ta sẽ cùng "mổ xẻ" một trường hợp thực tế đã được công bố công khai: nền tảng PRINCE (Preclinical Information Center) mà tập đoàn dược phẩm Bayer xây dựng cùng Thoughtworks, nhằm giúp các nhà khoa học truy vấn hàng thập kỷ báo cáo nghiên cứu tiền lâm sàng phần lớn nằm im lìm trong hàng ngàn file PDF.

Nội dung sách được viết lại, diễn giải và mở rộng dựa trên bài viết kỹ thuật gốc của tác giả Sarang Sanjay Kulkarni, đăng trên martinowler.com. Mục tiêu không phải là dịch nguyên văn, mà là chuyển hoá các bài học kỹ thuật đó thành một tài liệu tiếng Việt dễ tiếp cận vừa đủ sâu để một kỹ sư phần mềm có thể áp dụng, vừa đủ rõ ràng để một người quản lý sản phẩm hoặc một người mới tìm hiểu AI vẫn theo kịp mạch câu chuyện.

Sách được chia làm sáu phần. Phần một đặt bối cảnh bài toán và hành trình tiến hoá của hệ thống. Phần hai giới thiệu kiến trúc tổng thể và hai khái niệm xuyên suốt: kỹ thuật ngữ cảnh (context engineering) và kỹ thuật giàn giáo (harness engineering). Phần ba đi sâu vào từng "tác nhân" (agent) bên trong hệ thống. Phần bốn bàn về niềm tin thứ tài sản quý giá nhất của bất kỳ hệ thống AI nào. Phần năm nói về khả năng phục hồi khi có sự cố. Và phần sáu khép lại bằng những bài học rút ra cho bất kỳ ai đang hoặc sắp xây dựng hệ thống agentic AI của riêng mình.

Cuốn sách này được biên soạn và chia sẻ miễn phí cho cộng đồng, với hy vọng góp một viên gạch nhỏ vào kho tri thức tiếng Việt về AI ứng dụng trong thực tế.

Chương 1 Vì sao ngành dược phẩm cần một trợ lý AI biết đọc PDF

Hãy tưởng tượng bạn là một nhà khoa học độc chất học (toxicology) tại một tập đoàn dược phẩm lớn. Trước khi một hợp chất mới được đưa vào thử nghiệm lâm sàng trên người, nó phải trải qua vô số nghiên cứu tiền lâm sàng kiểm tra độ an toàn, liều lượng, tác dụng phụ trên động vật thí nghiệm. Mỗi nghiên cứu như vậy tạo ra một báo cáo dài hàng trăm trang, đầy bảng biểu, thuật ngữ chuyên ngành, và đôi khi được quét lại từ bản giấy có tuổi đời hàng chục năm.

Giờ hãy nhân con số đó lên hàng chục ngàn báo cáo, tích lũy qua nhiều thập kỷ, nằm rải rác trong nhiều hệ thống lưu trữ khác nhau do lịch sử sáp nhập, di dời dữ liệu và thay đổi công nghệ. Khi một nhà nghiên cứu cần trả lời câu hỏi như "hợp chất X có từng gây ra hiện tượng co giật ở chuột cống trong bất kỳ thử nghiệm nào trước đây không?", họ buộc phải lục tìm thủ công qua rất nhiều tài liệu một công việc tốn thời gian, dễ bỏ sót, và kéo nhà khoa học ra khỏi công việc chuyên môn thực sự của họ.

Điều đáng nói là bài toán này không phải là chuyện riêng của một công ty. Bất kỳ tổ chức nào có tuổi đời đủ lâu ngân hàng, công ty bảo hiểm, cơ quan chính phủ, tập đoàn sản xuất đều tích lũy một khối lượng khổng lồ tài liệu, hợp đồng, báo cáo, biên bản mà không ai đọc lại nữa, nhưng đôi khi lại chứa đựng câu trả lời cho một câu hỏi quan trọng hôm nay. Trong ngành dược, cái giá của việc "không tìm ra" thông tin đó có thể rất lớn: một nghiên cứu độc chất học cũ có thể đã ghi nhận một tác dụng phụ hiếm gặp, và nếu thông tin đó bị bỏ sót, nó có thể dẫn đến việc lặp lại một thử nghiệm tốn kém không cần thiết, hoặc tệ hơn, bỏ lỡ một tín hiệu an toàn quan trọng.

Đây chính là bài toán mà đội ngũ tại Bayer phải đối mặt, và nó có ba lớp khó khăn chồng lên nhau:

- Dữ liệu phân mảnh (data silos): thông tin về cùng một hợp chất hoặc nghiên cứu nằm rải rác ở nhiều hệ thống khác nhau, khó có được một bức tranh tổng thể.
- Tìm kiếm theo từ khóa không đủ thông minh: các công cụ tìm kiếm truyền thống dựa trên logic Boolean cứng nhắc, không xử lý tốt sự đa dạng và tinh tế của thuật ngữ khoa học.
- Phân tích thủ công tốn thời gian: việc tổng hợp thông tin từ nhiều tài liệu đòi hỏi công sức lớn, làm giảm thời gian dành cho nghiên cứu khoa học thực chất.

Sự xuất hiện của các mô hình ngôn ngữ lớn (LLM) và kỹ thuật truy xuất tăng cường sinh (Retrieval-Augmented Generation RAG) mở ra một hướng đi mới: thay vì bắt con người tìm kiếm bằng từ khóa, hãy để họ đặt câu hỏi bằng ngôn ngữ tự nhiên, và để hệ

thống tự tìm, đọc, và tổng hợp câu trả lời có căn cứ từ chính kho dữ liệu nội bộ. Đó chính là điểm khởi đầu cho hành trình xây dựng PRINCE.

Một ngày làm việc điển hình, trước và sau

Để cảm nhận rõ hơn sự khác biệt mà một hệ thống như PRINCE mang lại, hãy hình dung hai kịch bản. Trước khi có hệ thống: nhà khoa học nhận được câu hỏi từ đồng nghiệp về tiền sử an toàn của một hợp chất, mở ra ba, bốn hệ thống lưu trữ khác nhau, gõ nhiều biến thể từ khoá khác nhau vào từng hệ thống, tải về hàng chục file PDF nghi ngờ có liên quan, rồi dành phần lớn buổi chiều để đọc lướt qua từng file tìm đoạn thông tin liên quan nhiều lúc phát hiện ra rằng thông tin họ cần lại nằm trong một báo cáo mà từ khoá tìm kiếm ban đầu không hề bắt được, vì thuật ngữ mô tả trong báo cáo khác với thuật ngữ họ dùng để tìm.

Sau khi có hệ thống: nhà khoa học gõ câu hỏi bằng ngôn ngữ tự nhiên vào PRINCE, hệ thống xác định phạm vi tìm kiếm phù hợp, chạy song song nhiều chiến lược truy xuất, và trong vòng vài chục giây trả về một câu trả lời tổng hợp kèm trích dẫn đến từng đoạn văn bản nguồn. Nhà khoa học dành phần lớn thời gian còn lại để kiểm chứng nhanh vài trích dẫn quan trọng nhất, thay vì tự mình lục tìm từ đầu. Sự khác biệt không chỉ là tốc độ nó còn là việc giảm đáng kể khả năng bỏ sót thông tin quan trọng do giới hạn của tìm kiếm từ khoá thủ công.

Vì sao tìm kiếm từ khoá truyền thống không đủ

Để hiểu vì sao RAG lại tạo ra khác biệt lớn đến vậy, ta cần hiểu rõ giới hạn của công cụ tìm kiếm truyền thống. Một công cụ tìm kiếm dựa trên từ khoá và logic Boolean (AND, OR, NOT) yêu cầu người dùng phải đoán trước chính xác từ ngữ nào xuất hiện trong tài liệu. Nhưng ngôn ngữ khoa học lại cực kỳ đa dạng: cùng một hiện tượng "lông dựng" trên da động vật có thể được ghi là "piloerection", "lông xù", hay mô tả gián tiếp bằng một câu miêu tả dài dòng trong phần kết quả quan sát lâm sàng. Một truy vấn từ khoá cứng nhắc sẽ bỏ lỡ tất cả những cách diễn đạt khác nhau đó, trừ khi người dùng liệt kê được đầy đủ mọi biến thể điều gần như bất khả thi khi làm việc với hàng chục ngàn tài liệu viết bởi nhiều tác giả, nhiều phòng thí nghiệm, qua nhiều thập kỷ.

RAG giải quyết vấn đề này bằng cách chuyển bài toán từ "khớp từ khoá chính xác" sang "khớp ý nghĩa". Bằng cách biểu diễn văn bản dưới dạng vector số học (embedding) nắm bắt ngữ nghĩa, hệ thống có thể tìm ra những đoạn văn bản có ý nghĩa gần với câu hỏi, ngay cả khi từ ngữ dùng hoàn toàn khác nhau. Đây là bước nhảy vọt về khả năng, nhưng như chúng ta sẽ thấy ở chương 7 bản thân tìm kiếm ngữ nghĩa thuần túy cũng có những

điểm yếu riêng (ví dụ với mã số nghiên cứu, tên riêng, con số chính xác), và đó là lý do PRINCE kết hợp cả tìm kiếm ngữ nghĩa lẫn tìm kiếm từ khoá trong một pipeline lai.

Code4Life®

Chương 2 Hành trình ba giai đoạn: Tìm kiếm, Hỏi đáp, rồi Hành động

Một bài học lớn từ câu chuyện PRINCE là: đừng cố xây một hệ thống AI hoàn hảo ngay từ đầu. Thay vào đó, hãy tiến hoá nó qua từng giai đoạn, mỗi giai đoạn giải quyết một lớp giá trị mới cho người dùng. PRINCE đã đi qua ba giai đoạn rõ rệt.

Giai đoạn 1: Tìm kiếm (Search)

Giai đoạn đầu tiên tập trung vào việc gom các kho dữ liệu tiền lâm sàng rời rạc về một công thông tin thống nhất, cho phép người dùng lọc và tra cứu dựa trên siêu dữ liệu (metadata) có cấu trúc của các nghiên cứu ví dụ như loài động vật thử nghiệm, hợp chất, tuyến đường dùng thuốc. Đây là bước nền tảng: biến một mớ hỗn độn thành một nơi có thể tra cứu được, dù vẫn còn giới hạn ở các bộ lọc cứng.

Giai đoạn 2: Hỏi đáp (Ask)

Vấn đề là phần lớn tri thức giá trị nhất không nằm trong siêu dữ liệu có cấu trúc, mà nằm trong nội dung văn bản của chính các báo cáo PDF và do lịch sử di dời hệ thống nhiều lần, siêu dữ liệu đôi khi bị thiếu hoặc sai, trong khi bản PDF gốc luôn là "chuẩn vàng" đáng tin cậy nhất. Giai đoạn hai vì vậy ứng dụng RAG để cho phép người dùng đặt câu hỏi bằng ngôn ngữ tự nhiên và nhận câu trả lời được trích xuất trực tiếp từ nội dung các báo cáo, kể cả những bản scan lịch sử.

Giai đoạn 3: Hành động (Do)

Ở giai đoạn hiện tại, PRINCE không chỉ trả lời câu hỏi mà còn đóng vai trò một trợ lý nghiên cứu chủ động, có khả năng thực hiện các tác vụ phức tạp nhiều bước nhờ vào kiến trúc đa tác nhân (multi-agent). Hệ thống có thể điều phối nhiều bước truy xuất, tổng hợp, và thậm chí hỗ trợ soạn thảo các tài liệu phục vụ hồ sơ pháp lý/quy định (regulatory documents).

Bài học cốt lõi: đừng nhảy thẳng đến "AI làm mọi việc". Hãy đi từ Tìm kiếm → Hỏi đáp → Hành động. Mỗi bước xây trên nền tảng dữ liệu và niềm tin đã có từ bước trước.

Sự tiến hoá này không chỉ là vấn đề công nghệ, mà còn là chiến lược sản phẩm: mỗi giai đoạn mang lại giá trị đo lường được cho người dùng, đồng thời tích lũy dần niềm tin cần thiết để chuyển sang giai đoạn phức tạp và rủi ro hơn tiếp theo.

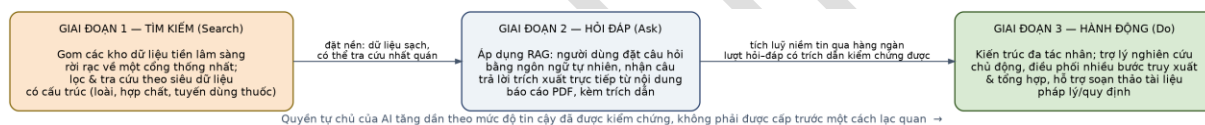
Vì sao thứ tự này quan trọng

Có một cám dỗ rất lớn khi bắt đầu một dự án AI mới: nhảy thẳng vào giai đoạn "Hành động", xây dựng ngay một agent có thể tự động làm mọi việc. Nhưng câu chuyện

PRINCE cho thấy một logic ngược lại và logic đó áp dụng được cho hầu hết các tổ chức. Giai đoạn Tìm kiếm buộc đội ngũ phải đối mặt và giải quyết vấn đề dữ liệu phân mảnh trước tiên: nếu dữ liệu còn rời rạc, sai lệch, hoặc không thể truy cập một cách nhất quán, thì không có mô hình AI nào dù mạnh đến đâu có thể tạo ra câu trả lời đáng tin cậy từ đó.

Giai đoạn Hỏi đáp sau đó xây dựng thói quen sử dụng và niềm tin ở mức độ rủi ro thấp: người dùng đặt câu hỏi, nhận câu trả lời kèm trích dẫn, và có thể tự kiểm chứng. Đây là giai đoạn để đội ngũ kỹ thuật học được rất nhiều điều cách người dùng thực sự đặt câu hỏi, những lỗi phổ biến của hệ thống truy xuất, những kỳ vọng chưa được đáp ứng mà không có rủi ro của việc để AI tự hành động thay con người.

Chỉ sau khi cả hai nền tảng đó đã vững dữ liệu sạch và có cấu trúc, cộng với niềm tin đã được xây dựng qua hàng ngàn lượt hỏi đáp thành công đội ngũ mới tiến đến giai đoạn Hành động, nơi hệ thống được trao quyền tự chủ nhiều hơn để thực hiện các tác vụ nhiều bước. Nói cách khác: quyền tự chủ của AI nên tỷ lệ thuận với mức độ tin cậy đã được kiểm chứng, không phải được cấp trước một cách lác quạn.



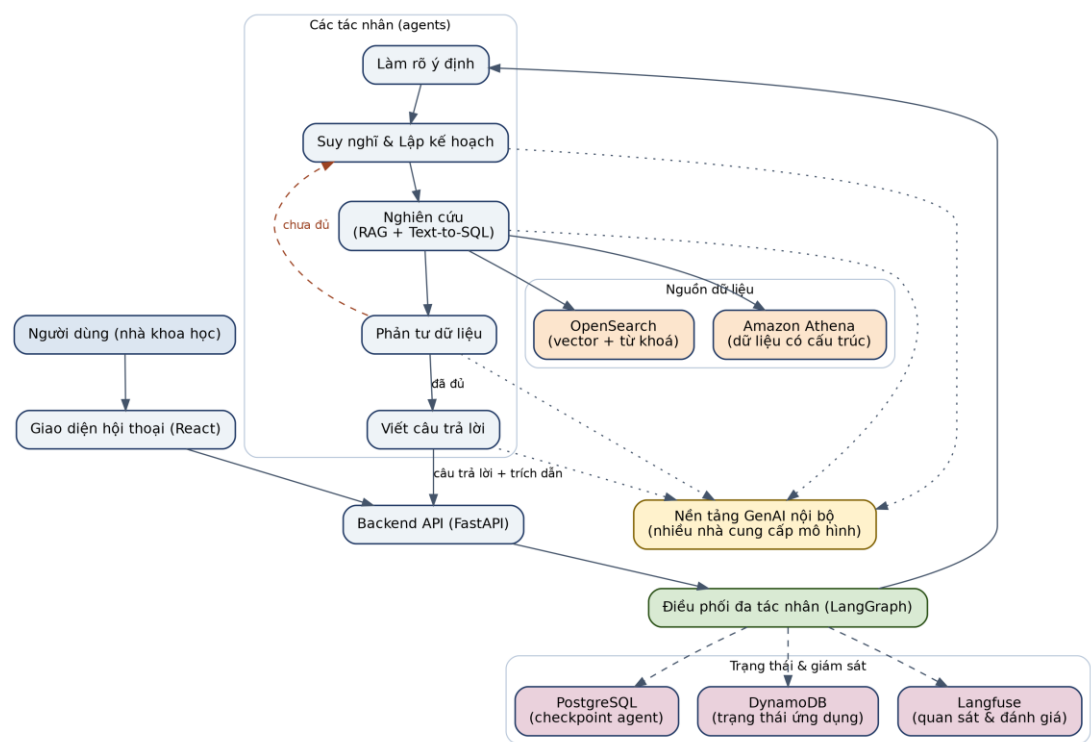
Sơ đồ 2.1 Ba giai đoạn tiến hoá của PRINCE: Tìm kiếm → Hỏi đáp → Hành động, quyền tự chủ của AI tăng dần theo mức độ tin cậy đã được kiểm chứng.

Chương 3 Bức tranh kiến trúc tổng thể

Về bản chất, PRINCE là một ứng dụng hội thoại: người dùng gõ câu hỏi vào một giao diện web (xây dựng bằng React), và phía sau là cả một dây chuyền xử lý phức tạp được điều phối bởi LangGraph một framework chuyên dùng để xây dựng luồng làm việc (workflow) cho các hệ thống agentic và được phục vụ qua một backend FastAPI.

Có thể hình dung luồng xử lý theo các lớp sau:

1. Yêu cầu người dùng: câu hỏi được gửi từ giao diện hội thoại.
2. Điều phối (Orchestration): lớp LangGraph tiếp nhận và dẫn dắt yêu cầu qua nhiều giai đoạn làm rõ ý định, suy nghĩ và lập kế hoạch, tiến hành nghiên cứu (kết hợp RAG và Text-to-SQL), kiểm tra tính đầy đủ của dữ liệu, và cuối cùng sinh câu trả lời.
3. Truy xuất dữ liệu: các tác nhân Nghiên cứu (Researcher) làm việc với một hệ sinh thái dữ liệu phân tán vector hoá nội dung báo cáo lưu trong OpenSearch làm kho tri thức chính; dữ liệu có cấu trúc được truy vấn qua Amazon Athena.
4. Quản lý trạng thái: sau mỗi bước xử lý, trạng thái của tác nhân được lưu lại trong PostgreSQL (thông qua cơ chế checkpoint của LangGraph), trong khi trạng thái ứng dụng ở tầng rộng hơn được lưu trong DynamoDB.
5. Mô hình ngôn ngữ: hệ thống sử dụng một nền tảng GenAI nội bộ, cho phép truy cập nhiều nhà cung cấp mô hình khác nhau (OpenAI, Anthropic, Google, và các mô hình mã nguồn mở) qua một giao diện tương thích chuẩn OpenAI, giúp dễ dàng hoán đổi mô hình phù hợp cho từng tác vụ.
6. Khả năng phục hồi: nếu một lệnh gọi tới mô hình thất bại, hệ thống tự động thử lại, và nếu vẫn thất bại thì chuyển sang mô hình/nhà cung cấp dự phòng khác.
7. Giám sát và đánh giá: CloudWatch theo dõi sức khoẻ hệ thống ở tầng hạ tầng, còn Langfuse đóng vai trò công cụ quan sát chính, ghi lại dấu vết (trace) chi tiết của toàn bộ lưu lượng thực tế, phục vụ gỡ lỗi và đánh giá chất lượng bằng framework RAGAS.



Sơ đồ 3.1 Kiến trúc tổng thể của PRINCE: từ câu hỏi người dùng, qua lớp điều phối đa tác nhân, đến các nguồn dữ liệu, mô hình ngôn ngữ, và hạ tầng lưu trạng thái/giám sát.

Để dễ hình dung, bảng dưới đây tóm tắt vai trò của từng thành phần công nghệ chính trong kiến trúc PRINCE:

Thành phần	Vai trò trong hệ thống
React (giao diện)	Giao diện hội thoại nơi người dùng đặt câu hỏi và xem câu trả lời kèm trích dẫn
LangGraph (điều phối)	Mô hình hoá và điều phối luồng xử lý đa tác nhân dưới dạng đồ thị có trạng thái
FastAPI (backend)	Phục vụ luồng xử lý qua API cho giao diện người dùng
Amazon OpenSearch	Kho vector lưu nội dung báo cáo đã nhúng, phục vụ tìm kiếm ngữ nghĩa và từ khoá
Amazon Athena	Truy vấn dữ liệu có cấu trúc đã qua chuẩn hoá (ETL)
PostgreSQL	Lưu trạng thái thực thi của các agent theo từng bước (checkpoint)
DynamoDB	Lưu trạng thái ứng dụng ở tầng rộng hơn: log, bước trung gian, trích dẫn
Langfuse	Quan sát (observability), lưu dấu vết thực thi và bộ dữ liệu đánh giá
CloudWatch	Giám sát sức khoẻ hệ thống ở tầng hạ tầng

Một nguyên tắc thiết kế xuyên suốt toàn bộ kiến trúc này là kỷ luật về ngữ cảnh (context discipline). Một phát hiện quan trọng trong quá trình phát triển là: cửa sổ ngữ cảnh (context window) của các mô hình ngôn ngữ ngày càng lớn hơn không có nghĩa là nên nhét mọi thứ vào một prompt duy nhất. Ngược lại, càng nhồi nhiều thông tin không chọn lọc vào ngữ cảnh, hệ thống càng khó kiểm soát và khó đánh giá. Vì vậy PRINCE thiết kế để mỗi giai đoạn trong luồng xử lý chỉ nhận đúng loại ngữ cảnh nó cần: ngữ cảnh lập kế hoạch cho bước Suy nghĩ & Lập kế hoạch, ngữ cảnh truy xuất cho tác nhân Nghiên cứu, ngữ cảnh bằng chứng cho tác nhân Phản tư, và ngữ cảnh tổng hợp cho tác nhân Viết.

Vì sao chọn kiến trúc đa tác nhân thay vì một mô hình duy nhất

Một câu hỏi tự nhiên mà bất kỳ ai mới tiếp cận agentic AI cũng sẽ đặt ra: tại sao không dùng một mô hình duy nhất, cực mạnh, để xử lý toàn bộ luồng công việc? Về lý thuyết điều đó khả thi, nhưng trong thực tế vận hành một hệ thống production, kiến trúc đa tác nhân mỗi tác nhân đảm nhận một trách nhiệm hẹp và rõ ràng mang lại nhiều lợi thế thiết thực. Thứ nhất, mỗi tác nhân có thể được đánh giá và cải thiện độc lập: khi chất lượng câu trả lời cuối cùng giảm sút, đội ngũ có thể khoanh vùng chính xác vấn đề nằm ở bước truy xuất, bước phản tư, hay bước viết, thay vì phải mò mẫm trong một hộp đen khổng lồ.

Thứ hai, mỗi tác nhân có thể dùng một mô hình khác nhau, tối ưu cho đúng loại công việc của nó: một mô hình nhỏ, nhanh, rẻ cho việc mở rộng câu hỏi; một mô hình mạnh về suy luận cho bước lập kế hoạch; một mô hình giỏi sinh mã cho Text-to-SQL. Thứ ba, việc chia nhỏ trách nhiệm giúp mỗi prompt ngắn gọn và tập trung hơn, giảm nguy cơ mô hình bị "phân tâm" bởi những chỉ dẫn không liên quan đến nhiệm vụ hiện tại đây chính là biểu hiện cụ thể của kỷ luật ngữ cảnh đã nói ở trên.

Vai trò của LangGraph như một "nhạc trưởng"

LangGraph đóng vai trò nhạc trưởng của toàn bộ dàn nhạc các tác nhân. Nó mô hình hoá luồng xử lý dưới dạng một đồ thị (graph), trong đó mỗi "nút" (node) là một bước xử lý có thể là một tác nhân, một lệnh gọi công cụ, hay một điểm quyết định rẽ nhánh. Cách tiếp cận dạng đồ thị này, thay vì một chuỗi lệnh tuyến tính đơn giản, cho phép hệ thống biểu diễn được những luồng xử lý phức tạp có vòng lặp (ví dụ vòng lặp phản tư quay lại truy xuất thêm dữ liệu), có điểm rẽ nhánh có điều kiện, và quan trọng nhất có thể tạm dừng ở bất kỳ đâu để chờ con người can thiệp hoặc để phục hồi sau sự cố.

Chương 4 Hai trụ cột: Kỹ thuật ngữ cảnh và Kỹ thuật giàn giáo

Nếu phải rút gọn toàn bộ triết lý kỹ thuật đằng sau PRINCE thành hai khái niệm, đó sẽ là context engineering (kỹ thuật ngữ cảnh) và harness engineering (kỹ thuật giàn giáo). Điều thú vị là khi hệ thống được thiết kế ban đầu, đội ngũ phát triển còn chưa dùng những thuật ngữ này chúng chỉ được đặt tên rõ ràng sau này, khi nhìn lại các quyết định kỹ thuật đã đưa ra.

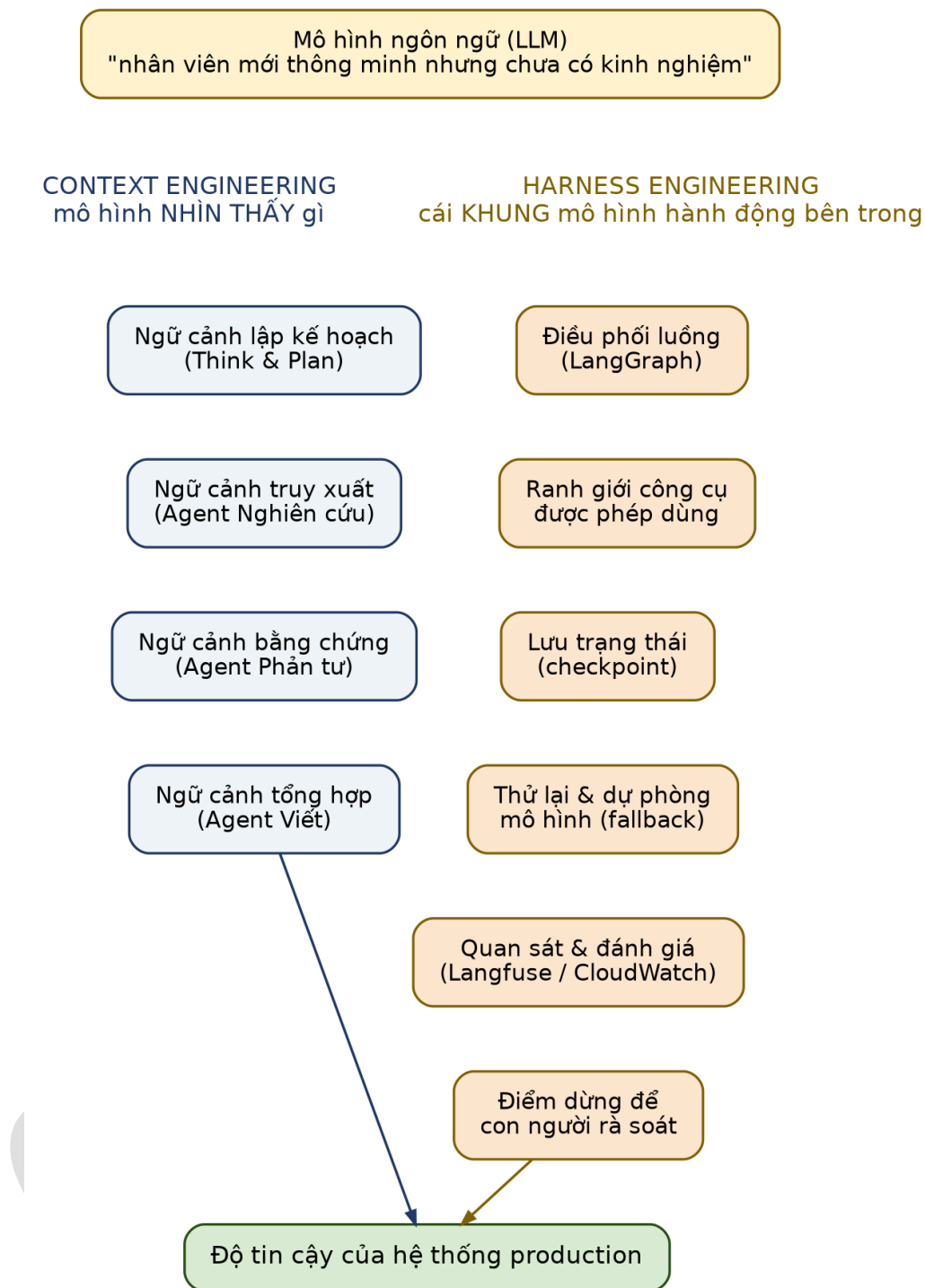
Kỹ thuật ngữ cảnh là gì?

Đây là nghệ thuật và kỹ thuật của việc định hình thông tin mà mỗi mô hình nhìn thấy: mô hình nhận được gì, không nhận được gì, và thông tin di chuyển như thế nào giữa các bước xử lý chuyên biệt như nghiên cứu, phản tư, và viết. Nói cách khác, đây không phải là việc "nhồi nhét cho đủ dữ liệu" mà là việc chọn lọc có chủ đích đưa đúng thông tin, đến đúng nơi, vào đúng thời điểm.

Kỹ thuật giàn giáo là gì?

Nếu context engineering lo về "cái gì mô hình nhìn thấy", thì harness engineering lo về "cái khung mà mô hình hành động bên trong". Đó là toàn bộ phần giàn giáo bao quanh mô hình: cách điều phối luồng công việc, ranh giới của các công cụ (tool) được phép dùng, cách lưu trạng thái, cơ chế thử lại khi lỗi, phương án dự phòng, việc xác thực kết quả, các vòng lặp phản tư, khả năng quan sát hệ thống, và các điểm dừng để con người xem xét.

Một mô hình ngôn ngữ mạnh không tự động tạo ra một hệ thống đáng tin cậy. Sự đáng tin cậy đến từ việc kỹ sư hoá cả hai lớp: những gì mô hình nhìn thấy, và cái khung mà mô hình hành động bên trong.



Sơ đồ 4.1 Hai trụ cột kỹ thuật của PRINCE: Context Engineering (mô hình nhìn thấy gì) và Harness Engineering (cái khung mô hình hành động bên trong), cùng hội tụ vào độ tin cậy của hệ thống.

Hai khái niệm này sẽ là sợi chỉ đỏ xuyên suốt phần còn lại của cuốn sách. Ở phần 3, chúng ta sẽ thấy context engineering thể hiện rõ nét qua cách từng tác nhân trong hệ thống chỉ nhận đúng phần thông tin cần thiết cho công việc của mình. Ở phần 5, chúng ta sẽ thấy harness engineering thể hiện qua các cơ chế lưu trạng thái, thử lại, và dự phòng mô hình.

Một phép ẩn dụ để hình dung

Nếu coi mô hình ngôn ngữ như một nhân viên mới cực kỳ thông minh nhưng chưa có kinh nghiệm, thì context engineering giống như việc chuẩn bị đúng tài liệu, đúng hồ sơ cần thiết trước khi giao việc cho nhân viên đó không đưa cho họ cả một tủ hồ sơ khổng lồ rồi bảo "tự tìm lấy", cũng không giấu bớt thông tin quan trọng khiến họ đoán mò. Còn harness engineering giống như việc thiết kế quy trình làm việc của công ty xung quanh nhân viên đó: ai duyệt việc gì, việc gì cần báo cáo lại, khi nào cần dừng lại hỏi ý kiến cấp trên, và điều gì xảy ra nếu họ mắc lỗi giữa chừng. Một nhân viên giỏi vẫn cần một quy trình làm việc tốt để phát huy hết năng lực một cách an toàn mô hình AI cũng vậy.

Vì sao hai khái niệm này thường bị bỏ qua

Trong giai đoạn đầu của làn sóng AI tạo sinh, phần lớn sự chú ý đổ dồn vào việc chọn mô hình nào tốt nhất, hay viết prompt nào khéo léo nhất. Đó là những yếu tố quan trọng, nhưng chúng chỉ là một phần của bài toán. Khi một hệ thống chuyển từ một demo ấn tượng sang một sản phẩm production phục vụ hàng trăm người dùng thực, phần lớn công sức kỹ thuật thực tế lại nằm ở việc thiết kế ngữ cảnh và giàn giáo những thứ ít "hào nhoáng" hơn nhưng lại quyết định trực tiếp đến việc hệ thống có đáng tin cậy hay không. Đây chính là bài học lớn nhất mà đội ngũ PRINCE rút ra sau nhiều năm vận hành hệ thống trong thực tế.

Chương 5 Bước đầu tiên: Làm rõ ý định người dùng

Một trong những sai lầm phổ biến nhất khi xây dựng hệ thống agentic là để mô hình tự đoán ý định người dùng rồi lao thẳng vào hành động kết quả là tốn kém chi phí tính toán cho những lần "thử sai" trên các công cụ và nguồn dữ liệu không phù hợp. PRINCE giải quyết vấn đề này bằng một bước gọi là Clarify User Intent (Làm rõ ý định người dùng), đóng vai trò tuyến phòng thủ đầu tiên chống lại sự mơ hồ.

Khi hệ thống mở rộng để bao phủ nhiều lĩnh vực khoa học khác nhau như độc chất học (toxicology) và dược lý học (pharmacology) những câu hỏi tương chừng đơn giản của người dùng lại trở nên mơ hồ, khiến việc tự động chọn đúng công cụ trở nên khó khăn. Thay vì để hệ thống thử hết mọi nguồn dữ liệu một cách tốn kém, PRINCE chủ động đặt câu hỏi làm rõ để xác định đúng lĩnh vực hoặc loại dữ liệu cần thiết trước khi tiến hành.

Ngoài việc hỏi lại người dùng, hệ thống còn có một cơ chế gợi ý nguồn dữ liệu thông minh: khi người dùng chưa chọn nguồn dữ liệu nào, hoặc chọn quá nhiều nguồn mà không rõ trọng tâm, mô hình sẽ phân tích ý định đằng sau câu hỏi và đề xuất những nguồn phù hợp nhất trong khi người dùng vẫn giữ toàn quyền chấp nhận, điều chỉnh, hoặc bác bỏ gợi ý đó. Đây là một nguyên tắc thiết kế quan trọng: chuyên môn của con người luôn có tiếng nói cuối cùng.

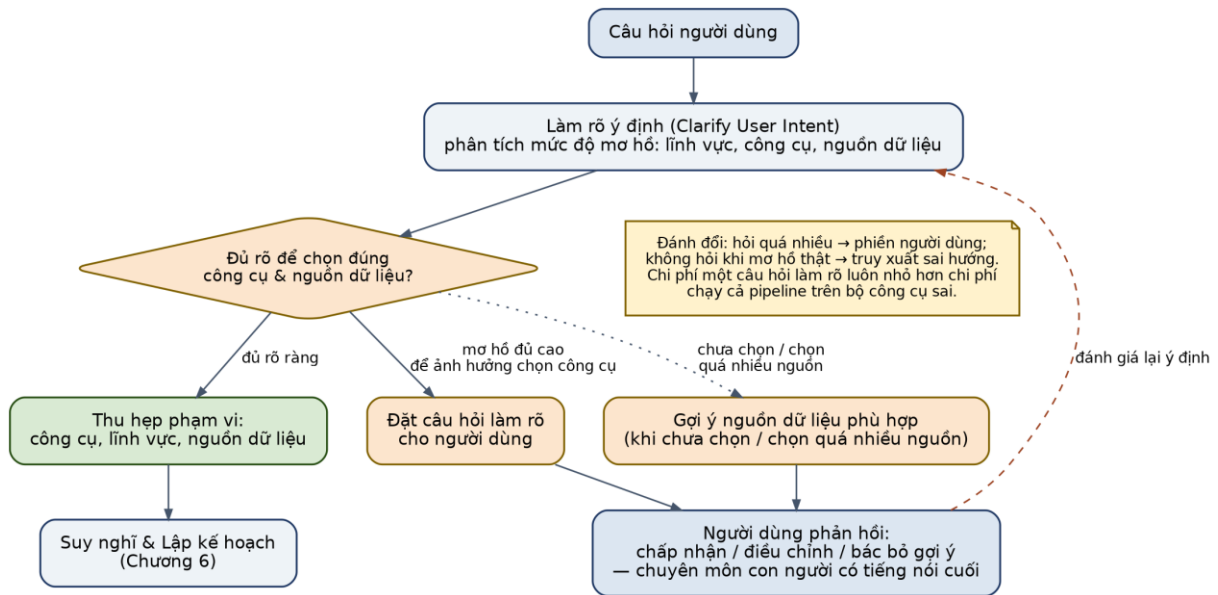
Xét theo góc độ kỹ thuật ngữ cảnh, bước này chính là quyết định "lắp ráp" đầu tiên trong toàn bộ luồng xử lý: nó giới hạn phạm vi công cụ, lĩnh vực, và nguồn dữ liệu sẽ được sử dụng trước khi bất kỳ hoạt động truy xuất nào diễn ra đảm bảo các tác nhân phía sau nhận được một bài toán đã được thu hẹp, thay vì một bài toán mở vô hạn.

Cân bằng giữa hỏi lại và làm phiền người dùng

Có một sự đánh đổi tinh tế ở đây: hỏi lại quá nhiều khiến người dùng cảm thấy phiền toái, như thể hệ thống không đủ thông minh để tự hiểu; nhưng không hỏi lại khi câu hỏi thực sự mơ hồ sẽ dẫn đến việc lãng phí thời gian và tài nguyên cho những lượt truy xuất sai hướng, và tệ hơn là đưa ra câu trả lời dựa trên một giả định sai về ý định người dùng. Đội ngũ PRINCE giải quyết đánh đổi này bằng cách tinh chỉnh cẩn thận ngưỡng "khi nào nên hỏi lại" hệ thống chỉ chủ động hỏi khi mức độ mơ hồ đủ cao để ảnh hưởng đến việc chọn công cụ, và giữ im lặng, tiến thẳng vào xử lý khi ý định đã đủ rõ ràng. Đây là một bài học chung cho bất kỳ ai thiết kế trải nghiệm hội thoại với AI: sự "thông minh" không nằm ở việc hỏi càng nhiều câu càng tốt, mà ở việc biết chính xác khi nào cần hỏi.

Bài học cho người xây dựng hệ thống của riêng bạn

Nếu bạn đang thiết kế một hệ thống agentic AI có nhiều nguồn dữ liệu hoặc nhiều công cụ chuyên biệt, hãy cân nhắc nghiêm túc việc đưa vào một bước làm rõ ý định tương tự, thay vì để mô hình "đoán" công cụ phù hợp ngay từ câu hỏi đầu tiên. Chi phí của một câu hỏi làm rõ thường nhỏ hơn rất nhiều so với chi phí của việc chạy toàn bộ pipeline truy xuất trên một bộ công cụ sai, rồi phải xử lý lại từ đầu.



Sơ đồ 5.1 Luồng làm rõ ý định người dùng (Clarify User Intent): từ phân tích mức độ mơ hồ đến quyết định hỏi lại hoặc thu hẹp phạm vi tiến thẳng vào xử lý.

Chương 6 Suy nghĩ trước khi hành động: bước Think & Plan

Hãy thử tưởng tượng một hệ thống cần thực hiện 50 bước liên tiếp để hoàn thành một tác vụ phức tạp. Nếu không có điểm dừng nào để tự kiểm tra, hệ thống rất dễ đi lạc hướng mà không hề hay biết. Đây chính là lý do PRINCE có một bước dành riêng cho việc suy nghĩ, được đặt tên là Think & Plan (Suy nghĩ & Lập kế hoạch) lấy cảm hứng từ kỹ thuật "think tool" mà Anthropic từng giới thiệu.

Điều quan trọng cần phân biệt: bước này thực hiện cái gọi là phản tư về quy trình (process reflection) tức là đánh giá xem tác nhân có đang đi đúng hướng, đúng tiến độ hay không, chứ không phải đánh giá bản thân dữ liệu đã thu thập được (việc đó thuộc về một tác nhân khác, sẽ nói ở chương sau).

Ở mỗi thời điểm then chốt, hệ thống tự hỏi: "Mình có đang thực hiện các bước theo đúng cách không? Mình có đang tiến triển đúng như kỳ vọng không? Hướng đi hiện tại có dẫn tới mục tiêu của người dùng không?" Khả năng "siêu nhận thức" (metacognition) này giúp hệ thống tự phản tư về chính luồng xử lý của nó và điều chỉnh chiến lược khi cần.

Vì sao bước này trở nên thiết yếu khi hệ thống có nhiều công cụ

Ban đầu PRINCE chỉ có hai công cụ: truy xuất RAG và truy vấn Text-to-SQL. Nhưng khi số lượng nguồn dữ liệu tăng lên, số công cụ cũng tăng theo và với nó là một vấn đề mới: các công cụ có phạm vi trùng lặp, khó phân biệt. Ví dụ, một công cụ truy vấn siêu dữ liệu có cấu trúc và một công cụ khác truy xuất báo cáo phi cấu trúc có thể trông tương tự nhau nhưng phục vụ những mục đích tinh tế khác nhau.

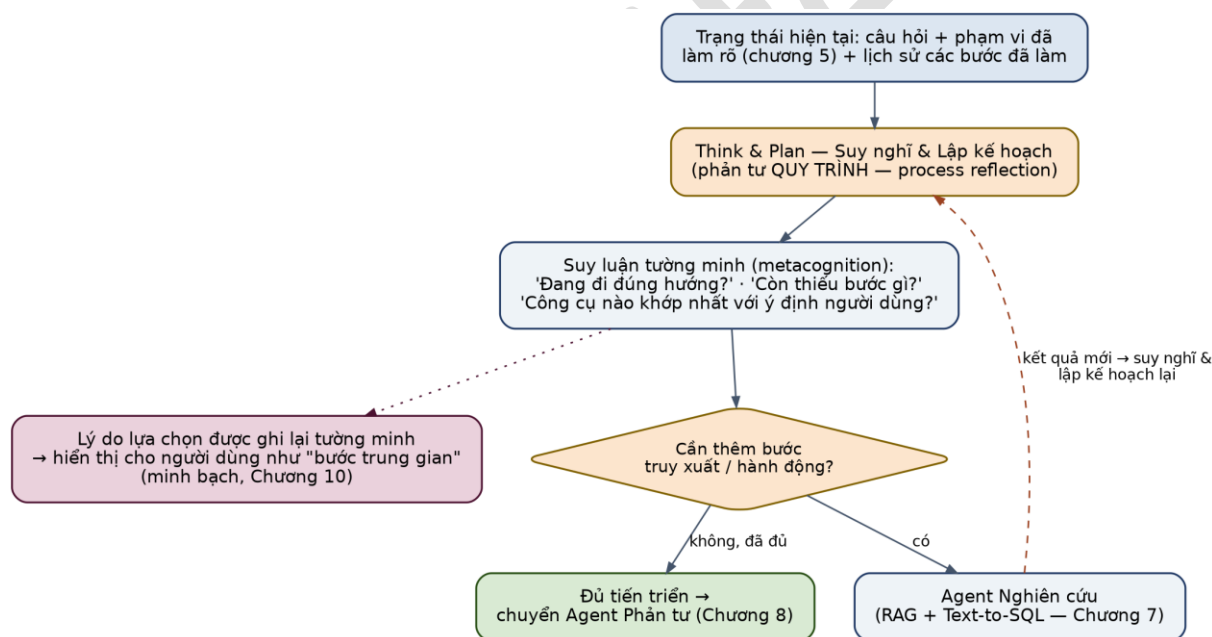
Bằng cách dành riêng một bước để suy nghĩ, hệ thống có thể lý giải một cách tường minh xem công cụ nào khớp nhất với ý định của người dùng, đánh giá đặc điểm của từng công cụ khả dụng, và đưa ra lựa chọn có cân nhắc điều này đã cải thiện đáng kể độ chính xác trong việc chọn công cụ.

Bước Think & Plan cũng đóng vai trò điều phối các quy trình nhiều bước: chẳng hạn hệ thống có thể trước tiên truy vấn siêu dữ liệu có cấu trúc để xác định các nghiên cứu liên quan, sau đó dùng mã số nghiên cứu đó để truy xuất chi tiết từ báo cáo phi cấu trúc, rồi cuối cùng tổng hợp lại. Nếu không có không gian phản tư này, hệ thống dễ thực hiện các bước một cách máy móc, tuyến tính, mà không đánh giá xem mỗi bước có thực sự đưa nó đến gần mục tiêu hay không.

Vì sao "không gian để suy nghĩ" lại hiệu quả đến vậy

Có một trực giác sai lầm phổ biến rằng mô hình ngôn ngữ chỉ cần đủ thông minh là có thể chọn đúng công cụ ngay lập tức, không cần bước trung gian nào. Nhưng thực tế vận hành cho thấy điều ngược lại: khi được yêu cầu "vừa quyết định vừa hành động" trong cùng một bước, mô hình dễ mắc phải những lựa chọn vội vàng, thiên về công cụ "nghe có vẻ đúng" dựa trên tên gọi, thay vì thực sự cân nhắc đặc điểm dữ liệu bên dưới. Việc tách một không gian riêng, chỉ để suy luận mà chưa hành động, buộc mô hình phải trình bày tường minh lý do lựa chọn của nó và chính hành động "viết ra lý do" đó, tương tự như kỹ thuật chuỗi suy luận (chain-of-thought) được biết đến rộng rãi, giúp cải thiện đáng kể chất lượng quyết định cuối cùng.

Ngoài ra, việc ghi lại tường minh quá trình suy luận này còn mang lại một lợi ích phụ quan trọng cho tính minh bạch: như chúng ta sẽ thấy ở chương 10, nội dung của bước Think & Plan chính là một phần trong những gì được hiển thị cho người dùng như các "bước trung gian", giúp họ hiểu được vì sao hệ thống lại chọn hướng tiếp cận như vậy.



Sơ đồ 6.1 Bên trong bước Think & Plan: vòng lặp suy luận tường minh (phản tư quy trình) quyết định có cần thêm bước truy xuất hay đã đủ tiến triển.

Chương 7 Agent Nghiên cứu: khi RAG bắt tay với SQL

Nếu Think & Plan là "bộ não chiến lược", thì Researcher Agent (Agent Nghiên cứu) chính là "đôi tay" đi thu thập thông tin thực tế. Đây là thành phần thu thập thông tin chính của hệ thống, và trải nghiệm phát triển PRINCE cho thấy một quy luật lặp lại: khi mở rộng sang lĩnh vực khoa học mới, dữ liệu luôn rơi vào hai loại lớn dữ liệu có cấu trúc và dữ liệu phi cấu trúc.

Khi PRINCE mở rộng ra nhiều lĩnh vực tiền lâm sàng, một Agent Nghiên cứu duy nhất với danh sách công cụ dần trở nên khó quản lý. Rất nhiều công cụ xoay quanh những khái niệm tương tự nhau "nghiên cứu", "phát hiện", "thử nghiệm" nhưng lại trở tới những bộ dữ liệu, lược đồ (schema), và cách diễn giải quy định khác nhau tùy theo lĩnh vực. Ví dụ, khi người dùng nhắc đến "nghiên cứu này", nó có thể là một nghiên cứu độc chất học liều lặp lại, một gói dữ liệu dược lý an toàn tim mạch, hay một thử nghiệm cụ thể trong bảng dữ liệu tổng hợp quy mô lớn mỗi loại có nguồn thông tin đáng tin cậy riêng.

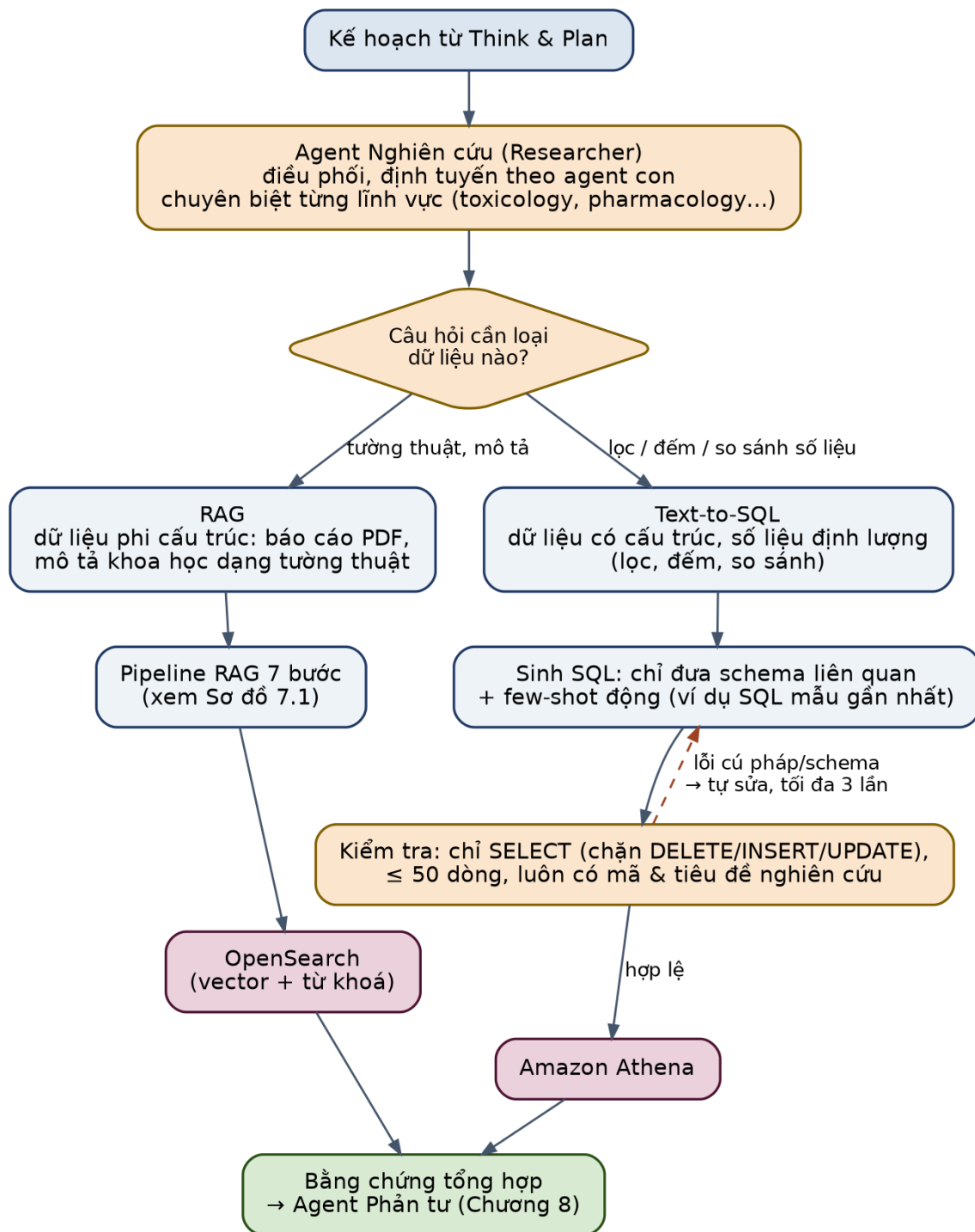
Để tránh tình trạng một agent "ôm đồm" quá nhiều công cụ có hợp đồng dữ liệu (data contract) khác nhau, đội ngũ phát triển đang chuyển dần Agent Nghiên cứu thành một cấu trúc phân cấp gồm các agent con chuyên biệt theo từng lĩnh vực mỗi agent con sở hữu bộ công cụ và chỉ dẫn prompt riêng, được tinh chỉnh theo cách dữ liệu của lĩnh vực đó vận hành. Agent Nghiên cứu ở tầng trên khi đó đóng vai trò điều phối viên, định tuyến câu hỏi đến đúng agent con phù hợp.

Hai chiến lược truy xuất song song

- RAG (Retrieval-Augmented Generation): dùng cho dữ liệu phi cấu trúc, chủ yếu là các báo cáo PDF.
- Text-to-SQL: dùng cho dữ liệu có cấu trúc, lưu trữ trong Amazon Athena.

Cách tiếp cận kép này cho phép hệ thống bắc cầu giữa những báo cáo khoa học dạng tường thuật và dữ liệu thực nghiệm định lượng. Bảng dưới đây tóm tắt nhanh khi nào nên dùng chiến lược nào:

Tiêu chí	RAG (truy xuất phi cấu trúc)	Text-to-SQL (dữ liệu có cấu trúc)
Loại dữ liệu phù hợp	Báo cáo PDF, văn bản tường thuật, mô tả khoa học	Bảng số liệu, kết quả thử nghiệm định lượng
Loại câu hỏi phù hợp	"Có phát hiện X trong nghiên cứu Y không?"	"Cho tôi 50 nghiên cứu trên chuột cống"
Kết quả trả về	Đoạn văn bản kèm trích dẫn nguồn	Bảng dữ liệu có cấu trúc, giới hạn số dòng
Rủi ro chính	Bỏ sót do cách diễn đạt khác biệt	SQL sai cú pháp hoặc sai lược đồ
Cơ chế giảm rủi ro	Mở rộng câu hỏi + tìm kiếm lại + rerank	Few-shot động + vòng lặp tự sửa lỗi



Sơ đồ 7.2 Agent Nghiên cứu định tuyến giữa hai chiến lược truy xuất song song: RAG cho dữ liệu phi cấu trúc, Text-to-SQL cho dữ liệu có cấu trúc.

Bên trong pipeline RAG: một dây chuyền bảy bước

Điều thú vị nhất trong toàn bộ kiến trúc PRINCE có lẽ là cách pipeline RAG được thiết kế tỉ mỉ, không chỉ đơn giản là "nhúng câu hỏi rồi tìm vector gần nhất". Hãy lấy ví dụ một câu hỏi thực tế: "Có phát hiện lâm sàng nào sau đây được ghi nhận trong nghiên cứu T123456-2 không: lòng dưng, mất phối hợp vận động, mắt khép hờ, và phân lỏng?"

1. Trích xuất từ khoá: một mô hình phân tích câu hỏi và trích ra các từ khoá quan trọng cho việc tìm kiếm theo từ khoá trong kho tài liệu.
2. Sinh bộ lọc siêu dữ liệu: song song đó, mô hình sinh ra một bộ lọc, ví dụ giới hạn theo đúng mã số nghiên cứu điều này giúp thu hẹp không gian tìm kiếm từ hàng triệu vector xuống chỉ còn vài chục đến vài trăm.
3. Mở rộng câu hỏi (query expansion): một mô hình nhỏ và nhanh hơn sinh ra năm biến thể diễn đạt khác nhau của cùng câu hỏi gốc, nhằm bắt được các cách dùng từ khác nhau có thể xuất hiện trong tài liệu gốc.
4. Tìm kiếm lai song song (hybrid retriever): với mỗi biến thể câu hỏi, hệ thống thực hiện đồng thời tìm kiếm theo ngữ nghĩa vector (kNN) và tìm kiếm theo từ khoá trên OpenSearch, với trọng số 0,7 cho kết quả ngữ nghĩa và 0,3 cho kết quả từ khoá tỷ lệ này được xác định qua thực nghiệm.
5. Tổng hợp và xếp hạng sơ bộ: kết quả từ năm lượt tìm kiếm song song được gộp lại, lấy điểm cao nhất cho mỗi đoạn văn bản (chunk) trùng lặp, giữ lại khoảng 20 đoạn tiềm năng.
6. Xếp hạng lại (rerank): một mô hình cross-encoder chuyên dụng (bge-reranker-large) đánh giá lại mức độ liên quan của từng đoạn so với câu hỏi gốc, chọn ra 7 đoạn phù hợp nhất.
7. Sinh prompt cuối và trả lời có trích dẫn: 7 đoạn được chọn kết hợp với câu hỏi gốc tạo thành prompt cuối cùng cho một mô hình lý luận mạnh, sinh câu trả lời kèm trích dẫn cụ thể trở về đúng đoạn tài liệu nguồn.



Sơ đồ 7.1 Bảy bước của pipeline RAG trong PRINCE, từ câu hỏi gốc đến câu trả lời có trích dẫn.

Toàn bộ quy trình này được giám sát liên tục bằng Langfuse để phục vụ việc quan sát, đo lường hiệu năng và chất lượng.

Text-to-SQL: khi câu hỏi cần con số chính xác, không phải đoạn văn

Với những câu hỏi đòi hỏi lọc, tổng hợp, hay so sánh số liệu chính xác ví dụ "cho tôi 50 nghiên cứu mẫu thực hiện trên chuột cống" RAG không phải công cụ phù hợp. Đây là lúc Text-to-SQL vào cuộc, với quy trình gồm các bước chính: phân tích ý định và xác định các điểm dữ liệu cần lấy; chỉ đưa vào ngữ cảnh phần lược đồ cơ sở dữ liệu thực sự liên quan (thay vì toàn bộ schema khổng lồ) để giảm độ phức tạp cho mô hình;

sử dụng kỹ thuật few-shot prompting đồng lấy các ví dụ SQL mẫu liên quan nhất từ một "tầng ngữ nghĩa" riêng để hướng dẫn mô hình sinh đúng cú pháp SQL theo phương ngữ Athena.

Câu truy vấn sinh ra sau đó được kiểm tra để đảm bảo chỉ chứa lệnh SELECT (các lệnh DELETE, INSERT, UPDATE bị chặn tuyệt đối vì lý do an toàn dữ liệu), luôn bao gồm các cột thiết yếu như mã số và tiêu đề nghiên cứu, và giới hạn không quá 50 dòng kết quả mỗi lần để tránh làm ngập phản hồi. Nếu truy vấn thất bại vì lỗi cú pháp hay lược đồ, thông báo lỗi cùng câu truy vấn gốc được đưa ngược lại cho mô hình để tự sửa quá trình này được thử tối đa ba lần trước khi báo thất bại.

Một chi tiết thú vị: đội ngũ từng thử thêm một bước để mô hình khác "rà soát" lại câu SQL đã sinh ra trước khi thực thi, nhưng sau đó loại bỏ bước này vì mô hình rà soát đôi khi gán cò sai cho những câu truy vấn hoàn toàn hợp lệ, làm giảm hiệu quả mà không mang lại lợi ích tương xứng về độ chính xác. Đây là một bài học thực tế: không phải cứ thêm một lớp "kiểm tra bằng AI" là luôn tốt hơn.

Vì sao phải mở rộng câu hỏi thành năm biến thể

Bước mở rộng câu hỏi (query expansion) đáng được nhìn kỹ hơn, vì nó giải quyết một vấn đề rất thực tế của tìm kiếm ngữ nghĩa: vector embedding của một câu hỏi không phải lúc nào cũng nằm gần vector của đoạn văn bản chứa câu trả lời, đặc biệt khi người hỏi và người viết báo cáo dùng những cách diễn đạt khác nhau cho cùng một khái niệm y sinh. Bằng cách sinh ra năm cách diễn đạt lại câu hỏi gốc mỗi cách nhấn vào những từ đồng nghĩa hoặc góc nhìn hơi khác nhau hệ thống tăng đáng kể xác suất ít nhất một trong số các biến thể đó "chạm" gần với cách diễn đạt trong tài liệu gốc. Đây là một kỹ thuật tương đối rẻ (vì dùng một mô hình nhỏ, nhanh) nhưng mang lại cải thiện đáng kể về độ bao phủ (recall) của bước truy xuất.

Vì sao cần bước xếp hạng lại (rerank) sau khi đã tìm kiếm lai

Một câu hỏi hợp lý khác: nếu đã kết hợp tìm kiếm ngữ nghĩa và từ khoá, tại sao còn cần thêm một bước xếp hạng lại? Lý do nằm ở sự khác biệt về mục tiêu giữa hai bước. Bước truy xuất ban đầu (tìm kiếm lai trên 5 biến thể câu hỏi) được tối ưu cho tốc độ và độ bao phủ nó cần quét nhanh qua một không gian vector lớn và trả về một tập ứng viên đủ rộng để không bỏ sót thông tin quan trọng. Nhưng chính vì tối ưu cho tốc độ, độ chính xác xếp hạng của bước này không cao bằng một mô hình cross-encoder chuyên dụng như bge-reranker-large, vốn có thể xem xét đồng thời cả câu hỏi và từng đoạn văn bản ứng viên một cách sâu sắc hơn nhiều (dù chậm hơn, nên chỉ khả thi khi áp dụng cho một tập nhỏ ứng viên đã được thu hẹp). Kết hợp hai bước truy xuất nhanh trên diện

rộng, rồi xếp hạng lại chính xác trên diện hẹp là một mẫu hình kiến trúc rất phổ biến và hiệu quả trong các hệ thống RAG chất lượng cao.

Code4Life®

Chương 8 Agent Phản tư: dữ liệu đã đủ để trả lời chưa?

Có một sự khác biệt tinh tế nhưng quan trọng giữa hai câu hỏi: "Mình có đang làm đúng quy trình không?" và "Mình đã có đủ thông tin để trả lời chưa?". Câu hỏi đầu thuộc về Think & Plan (chương 6). Câu hỏi sau thuộc về Reflection Agent (Agent Phản tư) thực hiện cái gọi là phản tư về dữ liệu (data reflection).

Hai loại phản tư này phục vụ những mục đích khác nhau nhưng đều thiết yếu: một hệ thống có thể thực hiện một quy trình hoàn toàn đúng đắn (tốt về mặt quy trình) nhưng vẫn thu thập không đủ dữ liệu để trả lời câu hỏi; ngược lại, nó cũng có thể có sẵn đủ dữ liệu nhưng lại không tiến triển hiệu quả qua luồng xử lý.

Agent Phản tư được kích hoạt khi bước Think & Plan nhận định rằng quá trình đã tiến triển đủ xa và sẵn sàng để đánh giá dữ liệu. Nó so sánh ngữ cảnh đã thu thập được với câu hỏi gốc của người dùng, xác định những khoảng trống hoặc thông tin còn thiếu. Nếu thông tin chưa đủ, Agent Phản tư sẽ sinh ra các câu hỏi tiếp theo cụ thể, gửi ngược lại cho bước Think & Plan để tiếp tục truy xuất bổ sung. Nếu thông tin đã đủ, luồng xử lý tiến sang bước tiếp theo.

Vòng lặp phản tư dữ liệu chính là cơ chế giúp hệ thống tự tinh chỉnh chiến lược tìm kiếm dựa trên kết quả ban đầu thay vì trả lời vội vàng với thông tin nửa vời.

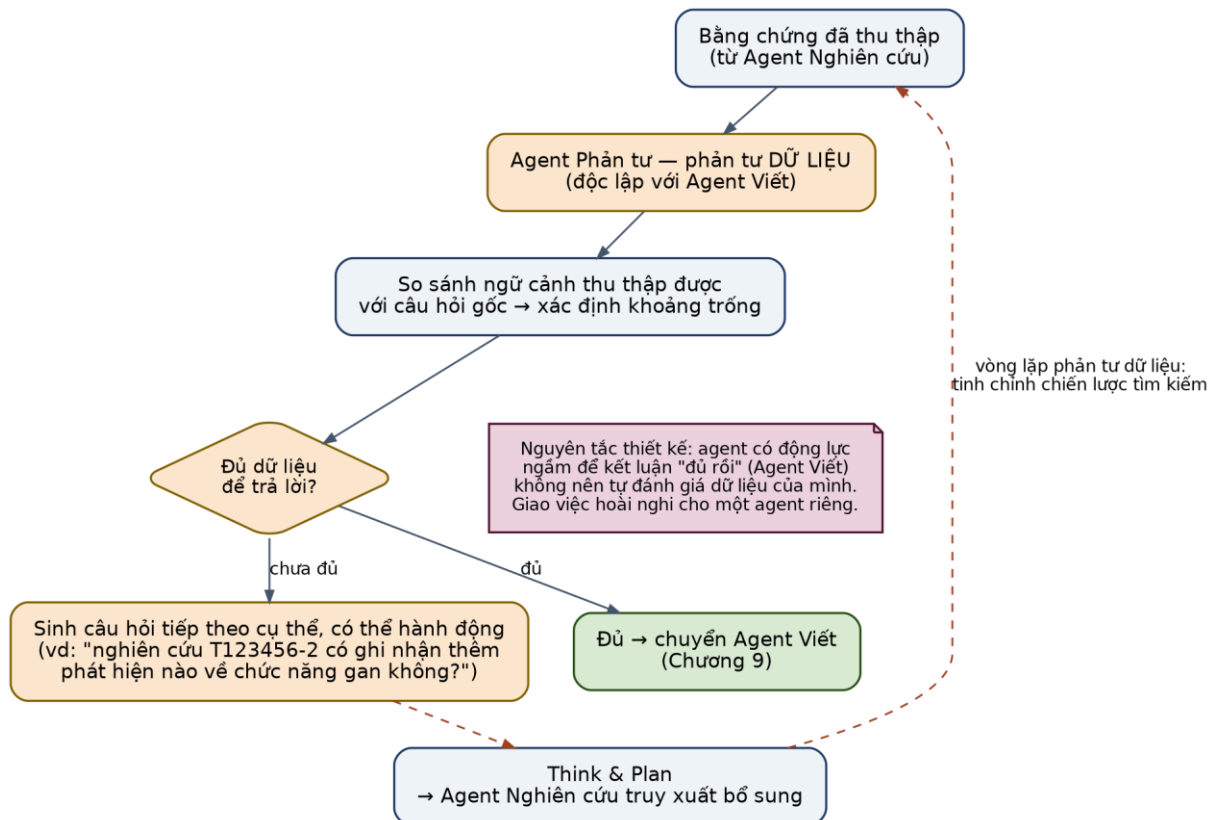
Vì sao "biết mình chưa biết đủ" lại khó với AI

Một trong những thách thức lớn nhất khi làm việc với mô hình ngôn ngữ là chúng có xu hướng luôn cố gắng tạo ra một câu trả lời, ngay cả khi thông tin sẵn có không đủ để trả lời chính xác. Hiện tượng này thường được gọi là ảo giác (hallucination). Agent Phản tư chính là cơ chế đối trọng trực tiếp với xu hướng đó: thay vì để Agent Viết tự quyết định "dữ liệu này đủ hay chưa" (một quyết định mà chính nó có động lực ngầm để trả lời "đủ rồi", vì nhiệm vụ của nó là sinh câu trả lời), PRINCE tách trách nhiệm này ra thành một agent độc lập, với nhiệm vụ duy nhất là hoài nghi và tìm ra khoảng trống.

Đây là một nguyên tắc thiết kế quan trọng có thể áp dụng rộng rãi: khi một agent có động lực ngầm để đưa ra một kết luận nhất định (ví dụ, agent Viết luôn muốn có đủ dữ liệu để hoàn thành nhiệm vụ viết của nó), đừng để chính agent đó tự đánh giá xem tiền đề của nó có vững chắc hay không. Hãy giao việc "phản biện" cho một agent khác, với một prompt được thiết kế riêng để tìm lỗi, tìm khoảng trống, thay vì để hoàn thành nhiệm vụ.

Follow-up questions: cầu nối giữa phản tư và hành động tiếp theo

Một điểm thiết kế đáng chú ý là Agent Phản tư không chỉ đơn giản trả về "đủ" hoặc "chưa đủ" nó sinh ra những câu hỏi tiếp theo cụ thể, có thể hành động được, thay vì một nhận xét chung chung như "cần thêm thông tin". Sự khác biệt này rất quan trọng: một câu hỏi cụ thể như "nghiên cứu T123456-2 có ghi nhận thêm phát hiện nào về chức năng gan không?" cho bước Think & Plan một mục tiêu rõ ràng để lập kế hoạch truy xuất tiếp theo, trong khi một nhận xét mơ hồ sẽ khiến bước tiếp theo phải tự đoán phải làm gì làm tăng nguy cơ lặp lại vòng lặp không hiệu quả.



Sơ đồ 8.1 Vòng lặp phản tư dữ liệu của Agent Phản tư: so sánh bằng chứng với câu hỏi gốc, sinh câu hỏi tiếp theo cụ thể khi dữ liệu chưa đủ.

Chương 9 Agent Viết: từ bằng chứng thô đến câu trả lời có trách nhiệm

Sau khi Agent Nghiên cứu đã thu thập đủ bằng chứng liên quan từ RAG và Text-to-SQL, công việc còn lại là biến "nguyên liệu thô" đó thành một câu trả lời hoàn chỉnh đây là nhiệm vụ của Writer Agent (Agent Viết). Điều quan trọng cần nhấn mạnh: công việc của Agent Viết không phải là "khám phá" thêm thông tin mới, mà là tổng hợp ngữ cảnh đã truy xuất được, tôn trọng chỉ dẫn của người dùng, và tuân thủ các ràng buộc chất lượng của hệ thống.

Agent Viết vận hành theo một số quy tắc không thể thương lượng: mọi khẳng định đưa ra phải có căn cứ trong ngữ cảnh được cung cấp, kèm trích dẫn chính xác trở về đúng đoạn văn bản và mã số nghiên cứu nguồn vì khả năng xác minh là yếu tố sống còn trong một môi trường được quản lý chặt chẽ như dược phẩm. Agent này cũng chịu trách nhiệm tuân theo các yêu cầu định dạng của người dùng (bảng biểu, gạch đầu dòng, cấu trúc theo từng phần) và các chuẩn trả lời đặc thù của từng lĩnh vực khoa học.

Với những câu trả lời phức tạp hơn như tóm tắt nhiều phần hoặc điền một phần vào mẫu tài liệu quy định kiến trúc hỗ trợ mở rộng Agent Viết bằng một vòng rà soát nội bộ ngắn: Agent Viết soạn nháp trước, sau đó một bước rà soát kiểm tra xem có thiếu phần nào, bảng biểu có mâu thuẫn không, có khoảng trống so với câu hỏi gốc không, và có thể gửi chỉ dẫn cụ thể để Agent Viết chỉnh sửa lại từng phần. Đây là một dạng phản tư nhẹ, tập trung vào tính đầy đủ và cách trình bày của câu trả lời bổ sung cho phần phản tư dữ liệu đã diễn ra trước đó.

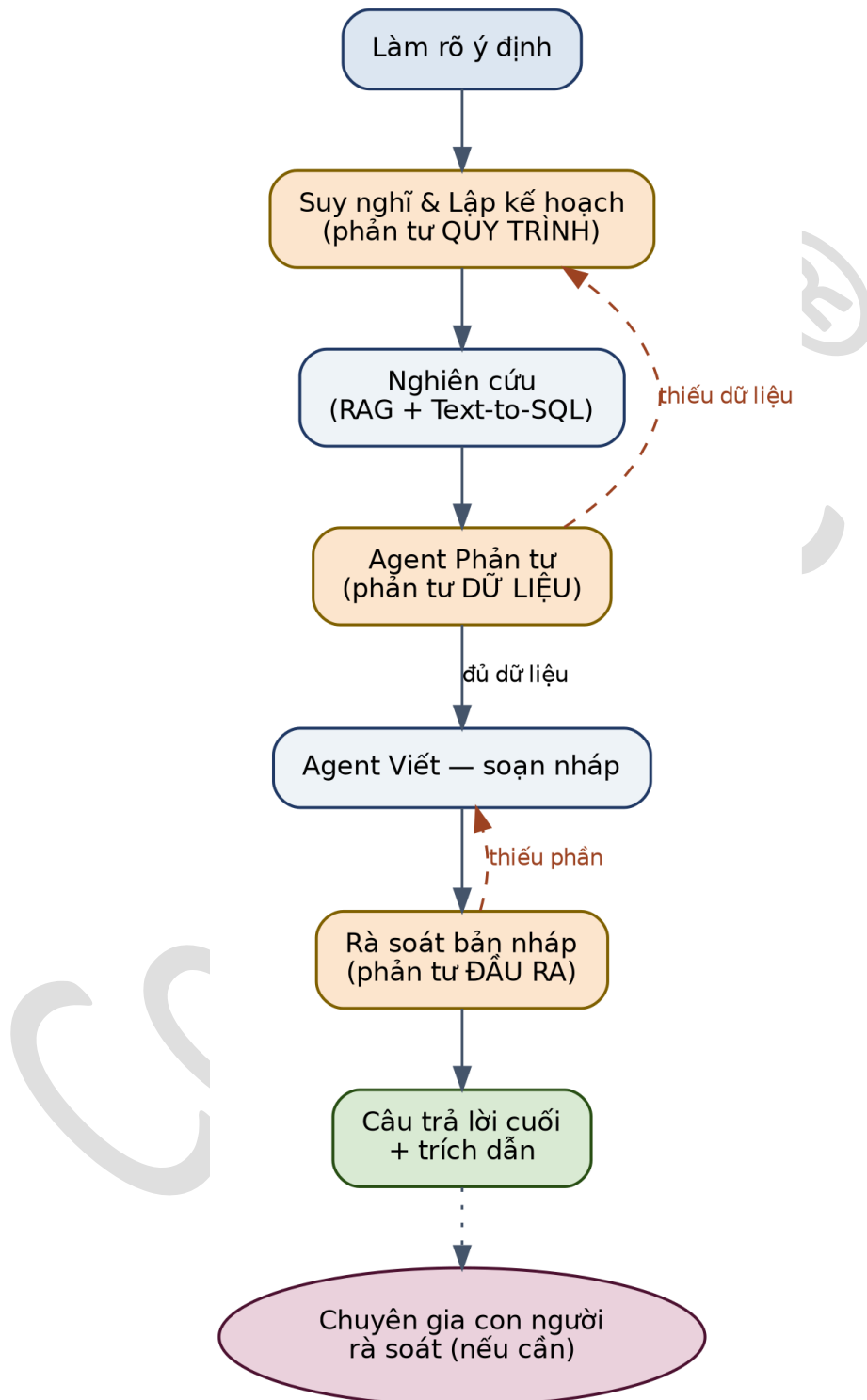
Một điểm rất quan trọng cần nhấn mạnh: mọi kết quả từ các luồng soạn thảo tài liệu quy định đều được thiết kế để phục vụ việc rà soát bởi chuyên gia con người bản đề trình cuối cùng luôn được soạn thảo và phê duyệt bởi nhân sự có đủ thẩm quyền chuyên môn. AI hỗ trợ, con người quyết định.

Ba vòng phản tư bổ sung cho nhau

Nhìn lại toàn bộ hành trình từ chương 6 đến chương 9, ta thấy PRINCE sở hữu ba vòng phản tư bổ sung lẫn nhau:

- Phản tư quy trình (Think & Plan): kiểm tra xem luồng xử lý có đi đúng hướng không, giúp phát hiện quỹ đạo sai, chọn sai công cụ, hoặc trình tự không hợp lý.
- Phản tư dữ liệu (Reflection Agent): kiểm tra xem bằng chứng thu thập được có đủ không, giúp phát hiện bằng chứng mông, thiếu ngữ cảnh, hoặc khoảng trống trong phạm vi bao phủ.

- Phản tư bản nháp (trong Writer Agent): kiểm tra xem kết quả sinh ra có đầy đủ không, giúp phát hiện phần bị thiếu, bảng biểu chưa hoàn chỉnh, hoặc lỗi hổng trong tổng hợp.



Sơ đồ 9.1 Toàn cảnh luồng xử lý PRINCE với ba vòng phản tư: quy trình, dữ liệu, và bản nháp đầu ra, kết thúc bằng khả năng rà soát của con người.

Cùng với nhau, các agent này thể hiện rõ nguyên tắc kỹ thuật ngữ cảnh đã nói ở chương 4: hệ thống không đơn giản là nhồi thêm thông tin vào một prompt khổng lồ, mà định tuyến đúng loại ngữ cảnh đến đúng năng lực, vào đúng thời điểm. Bước Text-to-SQL chỉ nhận phần lược đồ liên quan thay vì toàn bộ cơ sở dữ liệu; Agent Phản tư nhận câu hỏi gốc cùng bằng chứng đã thu thập để đánh giá khoảng trống, chứ không nhận toàn bộ lịch sử luồng xử lý; Agent Viết nhận các đoạn văn bản đã chọn lọc kèm ràng buộc trích dẫn, chứ không nhận dữ liệu truy xuất thô. Việc chuyển từ một agent "ôm đồm tất cả" sang một luồng xử lý có cấu trúc rõ ràng như vậy giúp mỗi agent có thể được đánh giá, gỡ lỗi, và cải thiện một cách độc lập.

Vì sao trích dẫn không phải là một tính năng "gắn thêm"

Có một cách tiếp cận dễ mắc phải khi xây dựng tính năng trích dẫn: sinh câu trả lời trước, rồi dùng một bước riêng để "tìm ngược" xem câu nào khớp với đoạn tài liệu nào. Cách làm này thường cho ra trích dẫn thiếu chính xác, vì mối liên hệ giữa câu trả lời và nguồn đã bị đứt gãy ngay từ lúc sinh văn bản. PRINCE tránh vấn đề này bằng cách yêu cầu Agent Viết gắn trích dẫn ngay trong quá trình sinh câu trả lời mỗi khẳng định được viết ra đồng thời với việc xác định rõ nó dựa trên đoạn ngữ cảnh nào. Đây là một quy tắc không thể thương lượng trong prompt của Agent Viết, chứ không phải một bước xử lý hậu kỳ có thể bỏ qua khi cần tối ưu tốc độ.

Chương 10 Niềm tin bắt đầu từ sự minh bạch

Trong một môi trường mà các quyết định có thể ảnh hưởng đến độ an toàn của thuốc và sức khỏe con người, niềm tin không phải là một tính năng "có thì tốt" nó là điều kiện tiên quyết để hệ thống được sử dụng. Và với một hệ thống AI, niềm tin không chỉ đến từ độ chính xác, mà còn từ khả năng minh bạch và cho phép người dùng tự kiểm chứng.

Hiển thị các bước trung gian

Vì luồng xử lý mang tính lặp và có thể tốn thời gian để ra kết quả cuối, việc duy trì tính minh bạch trong suốt quá trình là rất quan trọng. PRINCE hiển thị cho người dùng các bước trung gian mà hệ thống thực hiện các câu truy vấn được hình thành, các công cụ được sử dụng giúp người dùng theo dõi được lý do đằng sau câu trả lời cuối cùng, thay vì chỉ nhận một kết quả "từ trên trời rơi xuống".

Trích dẫn chi tiết đến từng câu

Đây có lẽ là cơ chế xây dựng niềm tin mạnh mẽ nhất trong toàn bộ hệ thống: mỗi câu trong câu trả lời sinh ra đều có thể được người dùng di chuột vào để xem trích dẫn tương ứng bao gồm liên kết đến tài liệu nguồn, số trang, và nguyên văn trích đoạn đã dùng để hỗ trợ cho phần trả lời đó. Mức độ chi tiết này không chỉ tăng độ tin cậy của hệ thống, mà còn giúp đơn giản hoá đáng kể quá trình rà soát của con người một yếu tố cực kỳ quan trọng trong ngành có quy định chặt chẽ như dược phẩm.

Nói cách khác, PRINCE không yêu cầu người dùng "tin tưởng mù quáng" vào AI. Nó cho họ công cụ để tự kiểm chứng và đó chính là nền tảng bền vững hơn nhiều so với việc chỉ cố gắng làm cho câu trả lời "nghe có vẻ đúng".

Sự khác biệt giữa "nghe có vẻ đúng" và "kiểm chứng được"

Đây là một phân biệt quan trọng mà bất kỳ ai xây dựng sản phẩm AI cũng nên khắc cốt ghi tâm. Một mô hình ngôn ngữ hiện đại có khả năng viết ra những câu trả lời rất trôi chảy, tự tin, và nghe có vẻ thuyết phục bất kể câu trả lời đó có chính xác hay không. Trong một môi trường ít rủi ro, điều này có thể chấp nhận được. Nhưng trong nghiên cứu tiền lâm sàng, nơi một kết luận sai có thể ảnh hưởng đến quyết định về độ an toàn của thuốc, sự tự tin trong giọng văn không có giá trị gì nếu không đi kèm khả năng kiểm chứng.

Chính vì vậy, thay vì đầu tư thêm công sức để làm cho câu trả lời "nghe thuyết phục hơn", đội ngũ PRINCE đầu tư vào việc làm cho câu trả lời "dễ kiểm chứng hơn". Đây

là một lựa chọn thiết kế có chủ đích: thừa nhận rằng mô hình có thể sai, và xây dựng toàn bộ trải nghiệm xoay quanh việc giúp con người phát hiện ra khi nào điều đó xảy ra, một cách nhanh chóng và ít tốn công sức nhất có thể.

Điều này cũng phản ánh một sự thay đổi tư duy quan trọng khi thiết kế sản phẩm AI cho môi trường doanh nghiệp có rủi ro cao: mục tiêu không phải là tạo ra một hệ thống "không bao giờ sai" một tiêu chuẩn gần như bất khả thi với công nghệ hiện nay mà là tạo ra một hệ thống mà khi nó sai, con người có thể phát hiện ra điều đó nhanh và dễ dàng. Đây là một tiêu chuẩn thực tế hơn nhiều, và cũng phù hợp hơn với bản chất xác suất vốn có của các mô hình ngôn ngữ hiện nay.

Hiệu ứng phụ: trích dẫn tốt giúp rút ngắn thời gian rà soát

Một lợi ích thường bị đánh giá thấp của cơ chế trích dẫn chi tiết là tác động của nó đến tốc độ làm việc của con người. Nếu không có trích dẫn, một nhà khoa học muốn kiểm chứng một câu trả lời AI sẽ phải tự mình tìm lại trong kho tài liệu một công việc tốn thời gian không kém gì việc tự tìm câu trả lời từ đầu, khiến cho lợi ích của AI gần như bị triệt tiêu. Nhưng khi mỗi câu đều có sẵn liên kết trực tiếp đến đúng trang, đúng đoạn văn bản nguồn, việc kiểm chứng chỉ mất vài giây thay vì vài phút hoặc vài giờ. Đây chính là lý do vì sao đội ngũ PRINCE coi trích dẫn granular (chi tiết đến từng câu) là một khoản đầu tư kỹ thuật xứng đáng, dù nó đòi hỏi thêm công sức trong thiết kế pipeline.

Chương 11 Đánh giá và giám sát: đo lường điều không thể thấy bằng mắt thường

Một hệ thống AI production không thể chỉ dựa vào cảm nhận "trông có vẻ ổn". PRINCE áp dụng hai loại đánh giá bổ trợ cho nhau.

Đánh giá theo bộ dữ liệu chuẩn (Dataset Evaluations)

Được thực hiện mỗi khi có thay đổi đáng kể về luồng xử lý cốt lõi, prompt, hoặc mô hình nền tảng. Các chuyên gia trong lĩnh vực chuẩn bị những bộ câu hỏi kèm đáp án tham chiếu, lưu trong Langfuse. Một kịch bản đánh giá tự động so sánh câu trả lời sinh ra với đáp án tham chiếu, tính toán các chỉ số định lượng như:

- Độ trung thực (Faithfulness): câu trả lời có được hỗ trợ đúng bởi ngữ cảnh hay không.
- Độ liên quan của câu trả lời (Answer Relevancy): câu trả lời có thực sự giải quyết câu hỏi hay không.
- Độ liên quan của ngữ cảnh (Context Relevancy): các đoạn được truy xuất có phù hợp hay không.
- Độ chính xác câu trả lời (Answer Accuracy): so sánh với đáp án chuẩn.
- Độ tương đồng ngữ nghĩa (Semantic Similarity): so với câu trả lời tham chiếu.

Vì bản chất agentic của hệ thống, việc áp dụng các chỉ số đánh giá phù hợp ở từng giai đoạn của luồng xử lý tương tự như "kim tự tháp kiểm thử" (testing pyramid) trong phát triển phần mềm truyền thống là điều cần thiết, bên cạnh việc đánh giá hiệu năng tổng thể đầu-cuối.

Đánh giá trên lưu lượng thực tế (Live Traffic Evaluations)

Được thực hiện hàng ngày dưới dạng một tác vụ batch trên các câu hỏi thực tế từ người dùng thật (không có đáp án tham chiếu trước). Các chỉ số như độ trung thực và độ liên quan vẫn có thể được đánh giá theo cách này. Đây là công cụ thiết yếu để giám sát hành vi hệ thống trong thực tế, phát hiện sớm các dấu hiệu ảo giác (hallucination), và hiệu hiệu năng trên sự đa dạng của câu hỏi thực tế.

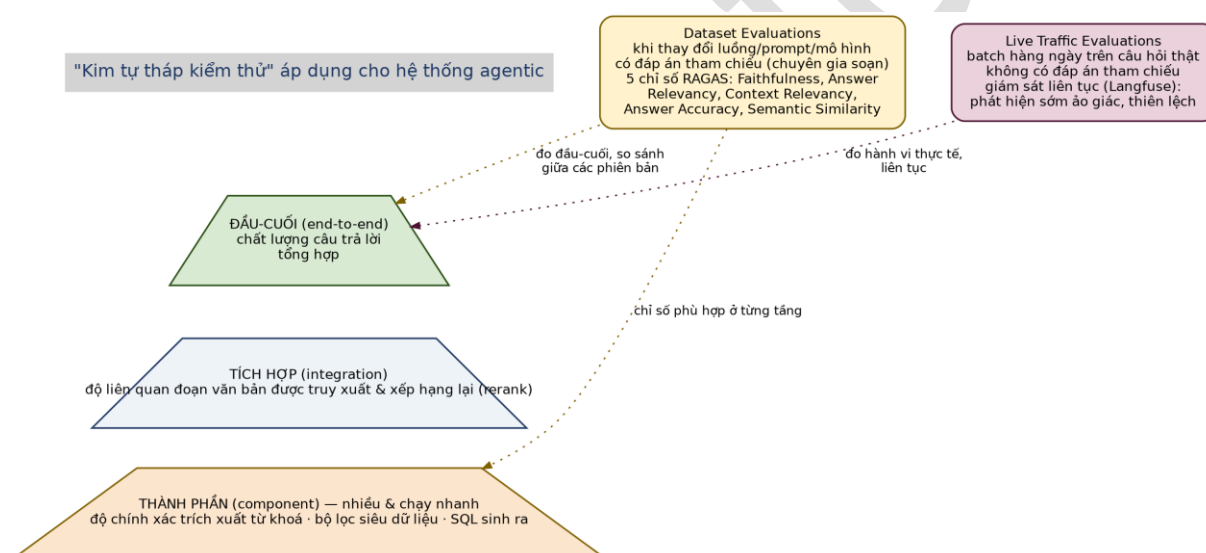
Giám sát liên tục

Song song với việc đánh giá định kỳ, Langfuse còn được dùng để giám sát liên tục nhằm phát hiện sớm các thiên lệch (bias), lỗi, hoặc các điểm cần cải thiện góp phần đảm bảo độ tin cậy và an toàn của câu trả lời hệ thống đưa ra theo thời gian.

"Kim tự tháp kiểm thử" áp dụng cho hệ thống agentic

Những ai từng làm việc trong phát triển phần mềm truyền thống sẽ quen thuộc với khái niệm kim tự tháp kiểm thử (testing pyramid): nhiều unit test nhỏ, chạy nhanh ở đáy; ít test tích hợp hơn ở giữa; và rất ít test đầu-cuối (end-to-end) ở đỉnh, vì chúng chậm và tốn kém. PRINCE áp dụng một logic tương tự cho việc đánh giá hệ thống agentic: thay vì chỉ đo lường "câu trả lời cuối cùng có đúng không", đội ngũ đánh giá riêng biệt từng tầng độ chính xác của bước trích xuất từ khoả, độ liên quan của các đoạn được truy xuất, độ chính xác của SQL được sinh ra, và cuối cùng mới đến chất lượng câu trả lời tổng hợp.

Cách tiếp cận phân tầng này mang lại một lợi ích lớn: khi chất lượng đầu ra cuối cùng giảm sút, đội ngũ có thể nhanh chóng xác định vấn đề nằm ở tầng nào có thể là bộ xếp hạng lại đang chọn sai đoạn văn bản, dù bước sinh câu trả lời vẫn hoàn toàn chính xác với dữ liệu nó nhận được. Nếu chỉ đo lường ở tầng đầu-cuối, đội ngũ sẽ biết "có gì đó sai" nhưng không biết sai ở đâu và việc gỡ lỗi một hệ thống nhiều tầng mà không có khả năng quan sát từng tầng là một cơn ác mộng thực sự.



Sơ đồ 11.1 “Kim tự tháp kiểm thử” áp dụng cho hệ thống agentic, cùng hai loại đánh giá bổ trợ: *Dataset Evaluations* và *Live Traffic Evaluations*.

Vì sao cần cả hai loại đánh giá, không chỉ một

Đánh giá theo bộ dữ liệu chuẩn có ưu điểm là đáng tin cậy, có thể lặp lại, và cho phép so sánh trực tiếp giữa các phiên bản hệ thống khác nhau nhưng nhược điểm của nó là bộ câu hỏi chuẩn không bao giờ phản ánh được hết sự đa dạng của câu hỏi thực tế mà người dùng đặt ra. Ngược lại, đánh giá trên lưu lượng thực tế phản ánh đúng những gì đang thực sự xảy ra, nhưng vì không có đáp án tham chiếu, nó chỉ đo được một số khía cạnh nhất định (như độ trung thực, không đo được độ chính xác so với đáp án đúng). Kết hợp cả hai loại đánh giá bù đắp cho điểm yếu của nhau: một loại cho biết "chúng ta

có đang cải thiện so với phiên bản trước không", loại kia cho biết "hệ thống đang hoạt động thế nào trong thế giới thực, ngay lúc này".

Code4Life®

Chương 12 Khi mọi thứ đi sai: xử lý lỗi và phục hồi trạng thái

Trong một luồng xử lý nhiều bước phức tạp, việc gặp lỗi là điều không thể tránh khỏi. Vấn đề không phải là "làm sao để không bao giờ lỗi" mà là "làm sao để lỗi không phá huỷ toàn bộ trải nghiệm người dùng". Đây chính là trọng tâm của harness engineering trong PRINCE.

Lưu trạng thái bền vững

Toàn bộ trạng thái của đồ thị luồng xử lý (workflow graph) được lưu trữ bền vững, cho phép hệ thống tiếp tục thực thi ngay từ node bị lỗi thay vì phải chạy lại từ đầu. Trạng thái Agent (tiến trình của các agent qua luồng xử lý) được lưu trong PostgreSQL; các khía cạnh khác của trạng thái ứng dụng như log, các bước trung gian, và trích dẫn được lưu trong DynamoDB. Việc tách biệt và lưu trữ bền vững này là yếu tố then chốt để đạt được độ ổn định trong một hệ thống agentic có trạng thái phức tạp.

Thử lại tự động và thử lại theo yêu cầu người dùng

Hệ thống được cấu hình sẵn cơ chế thử lại tự động ở nhiều bước trong luồng xử lý khi gặp lỗi tạm thời. Ngoài ra, người dùng cũng có thể chủ động yêu cầu thử lại một truy vấn thất bại qua giao diện. Khi đó, hệ thống tận dụng trạng thái đã lưu để tiếp tục ngay từ điểm lỗi, thông minh bỏ qua những bước đã hoàn thành thành công ở lần thử trước. Điều này vừa cải thiện trải nghiệm người dùng, vừa tiết kiệm đáng kể tài nguyên tính toán.

Dự phòng mô hình (LLM Fallbacks)

Để tăng cường độ tin cậy và giảm thiểu rủi ro liên quan đến việc mô hình ngưng hoạt động hoặc hiệu năng suy giảm, hệ thống có cơ chế dự phòng tùy chỉnh: nếu lệnh gọi đến nhà cung cấp hoặc mô hình chính thất bại sau vài lần thử, hệ thống tự động chuyển sang một mô hình thay thế từ nhà cung cấp khác. Cơ chế này đặc biệt quan trọng vì thời gian gián đoạn của các dịch vụ bên ngoài nằm ngoài tầm kiểm soát trực tiếp của đội ngũ phát triển.

Một chi tiết đáng chú ý: các agent còn được cung cấp ngữ cảnh về các lỗi đã xảy ra, để chúng có thể vạch ra một quỹ đạo khác hoặc kế hoạch hành động thay thế thay vì lặp lại y hệt bước đã thất bại.

Giàn giáo LangGraph đóng vai trò lớp kiểm soát bao quanh các agent: nó định nghĩa thành phần nào được phép hành động, được dùng công cụ gì, hệ thống có thể dừng ở đâu, lỗi được thử lại ra sao, trạng thái được lưu thế nào, và khi nào chuyển từ nghiên cứu sang phản tư rồi sang viết. Chính giàn giáo

này khiến hệ thống bớt "hộp đen" và đáng tin cậy hơn nhiều so với một agent tự trị không bị ràng buộc.

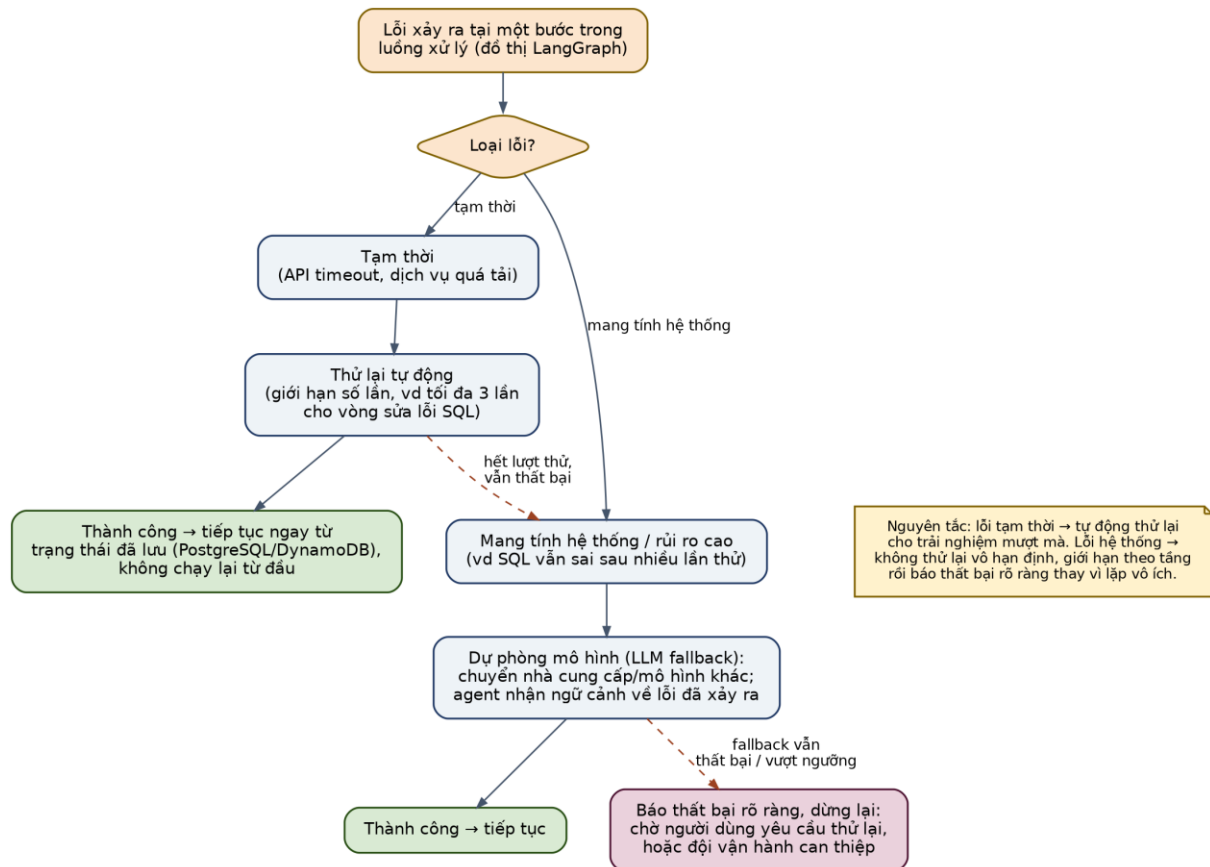
Chi phí và lợi ích của việc lưu trạng thái sau mỗi bước

Việc lưu trạng thái sau mỗi node trong đồ thị LangGraph không phải là miễn phí nó đòi hỏi thêm độ trễ (do phải ghi vào PostgreSQL) và thêm sự phức tạp trong việc thiết kế schema lưu trữ trạng thái. Nhưng cái giá đó nhỏ hơn rất nhiều so với chi phí của phương án thay thế: nếu không lưu trạng thái theo từng bước, một lỗi xảy ra ở bước thứ tám trong một luồng xử lý mười bước sẽ buộc toàn bộ luồng phải chạy lại từ đầu lãng phí tất cả các lệnh gọi mô hình đã thực hiện thành công ở bảy bước trước đó, cả về thời gian chờ đợi của người dùng lẫn chi phí tính toán thực tế.

Đây là một nguyên tắc harness engineering có thể tổng quát hoá: trong bất kỳ luồng xử lý nhiều bước nào có sử dụng lệnh gọi mô hình tốn kém, hãy lưu trạng thái đủ chi tiết để có thể tiếp tục từ điểm gần nhất có thể, thay vì chỉ lưu trạng thái ở đầu và cuối. Chi tiết càng nhỏ (theo từng node, thay vì theo từng phiên làm việc), khả năng phục hồi càng chính xác và tiết kiệm.

Khi nào nên tự động thử lại, và khi nào nên dừng lại chờ con người

Một quyết định thiết kế tinh tế khác là xác định ranh giới giữa "lỗi nên tự động thử lại" và "lỗi nên dừng lại và báo cho người dùng hoặc đội vận hành". Với các lỗi mang tính tạm thời như một lệnh gọi API bị timeout, hay một dịch vụ tạm thời quá tải thử lại tự động là hợp lý và mang lại trải nghiệm mượt mà cho người dùng. Nhưng với các lỗi mang tính hệ thống ví dụ một câu hỏi mà sau ba lần thử vẫn không sinh ra được một câu SQL hợp lệ việc tiếp tục thử lại vô hạn định không giải quyết được gì, chỉ làm chậm trải nghiệm và lãng phí tài nguyên. Đây là lý do PRINCE giới hạn số lần thử lại ở mỗi tầng (ví dụ tối đa ba lần cho vòng lặp sửa lỗi SQL), và khi vượt ngưỡng đó, hệ thống báo cáo thất bại một cách rõ ràng thay vì lặp lại vô ích.



Sơ đồ 12.1 Cây quyết định xử lý lỗi trong PRINCE: phân biệt lỗi tạm thời (thử lại tự động) với lỗi mang tính hệ thống (dự phòng mô hình, rồi dừng chờ con người).

Chương 13 Dữ liệu sạch hơn, hệ thống thông minh hơn

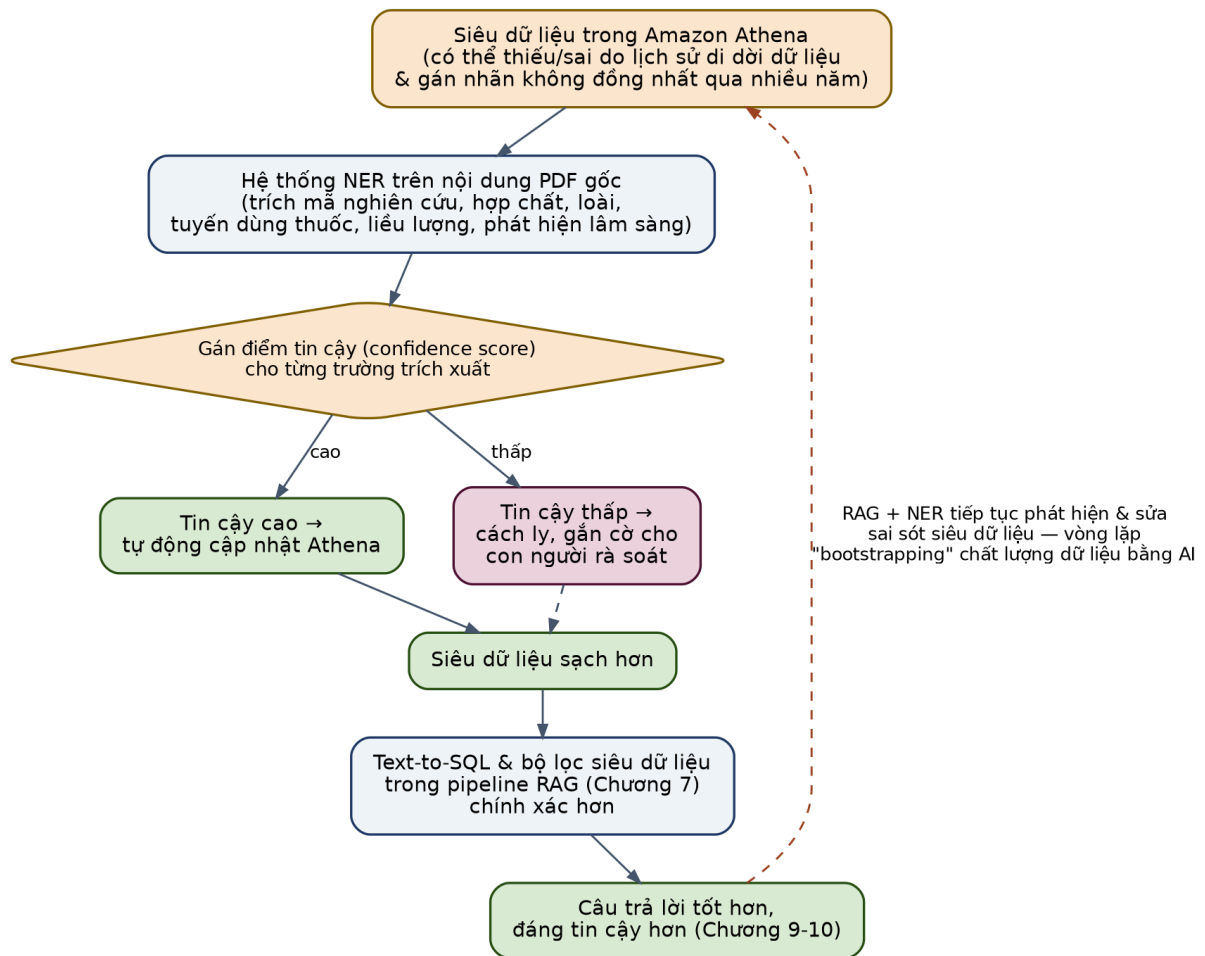
Chất lượng của Text-to-SQL và khả năng khám phá dữ liệu nói chung phụ thuộc trực tiếp vào độ chính xác và đầy đủ của siêu dữ liệu có cấu trúc trong Amazon Athena. Nhưng do lịch sử di dời dữ liệu nhiều lần và cách gán nhãn không đồng nhất giữa các phòng thí nghiệm và hệ thống khác nhau qua nhiều năm hoạt động của Bayer, siêu dữ liệu này đôi khi thiếu sót, sai lệch, hoặc không đầy đủ.

Để giải quyết vấn đề này, đội ngũ phát triển một hệ thống tiện ích sử dụng công nghệ Nhận dạng thực thể có tên (Named Entity Recognition NER) nhằm trích xuất và tạo ra các chú thích chính xác trực tiếp từ nội dung PDF của các báo cáo đọc văn bản và nhận diện các thực thể quan trọng như mã số nghiên cứu, tên hợp chất, loài động vật, tuyến đường dùng thuốc, thông tin liều lượng, và các phát hiện lâm sàng.

Điểm hay ở đây là cách hệ thống xử lý sự không chắc chắn: mỗi trường thông tin trích xuất được gán một điểm số tin cậy (confidence score). Những trường có độ tin cậy cao sẽ được tự động dùng để cập nhật cơ sở dữ liệu Athena; những trường có độ tin cậy thấp hơn sẽ được cách ly và gán cờ để con người rà soát và can thiệp. Đây là một ví dụ điển hình của nguyên tắc "tự động hoá những gì đáng tin cậy, giữ con người ở vòng lặp cho những gì chưa chắc chắn" một triết lý xuất hiện xuyên suốt toàn bộ hệ thống PRINCE, từ khâu trả lời câu hỏi cho tới khâu làm sạch dữ liệu nền.

Vòng lặp tự củng cố giữa chất lượng dữ liệu và chất lượng câu trả lời

Có một mối quan hệ hai chiều thú vị đáng được nhấn mạnh ở đây: chất lượng của siêu dữ liệu ảnh hưởng trực tiếp đến chất lượng của Text-to-SQL và cả bước lọc siêu dữ liệu trong pipeline RAG (chương 7); nhưng ngược lại, chính hệ thống RAG thông qua công nghệ NER áp dụng lên nội dung PDF lại là công cụ giúp phát hiện và sửa chữa những sai sót trong siêu dữ liệu đó. Nói cách khác, PRINCE không coi việc làm sạch dữ liệu là một dự án một lần rồi thôi, mà là một vòng lặp cải thiện liên tục, nơi chính năng lực AI của hệ thống được dùng để nâng cao chất lượng nền tảng dữ liệu mà nó dựa vào một dạng "bootstrapping" chất lượng dữ liệu bằng chính AI.



Sơ đồ 13.1 Vòng lặp tự củng cố giữa chất lượng siêu dữ liệu và chất lượng câu trả lời: NER trên PDF giúp làm sạch Athena, Athena sạch hơn giúp Text-to-SQL & RAG chính xác hơn.

Chương 14 Phát triển lặp: đừng chờ hoàn hảo mới ra mắt

PRINCE đã phục vụ người dùng thực tế từ đầu năm 2024, với phân tích hợp agentic được bổ sung vào cuối năm đó. Việc đưa hệ thống ra thực tế sớm thay vì chờ đến khi "hoàn hảo" chính là yếu tố then chốt giúp đội ngũ thu thập phản hồi thực tế và thúc đẩy quá trình phát triển lặp đi lặp lại.

Nguyên tắc chỉ đạo xuyên suốt là: xây dựng một ứng dụng LLM sẵn sàng cho production là một quá trình lặp, không phải một dự án có điểm kết thúc rõ ràng. Đội ngũ không chờ tính năng phải hoàn hảo tuyệt đối trước khi tìm kiếm phản hồi người dùng, mà ưu tiên mang lại giá trị sớm và liên tục tinh chỉnh dựa trên cách hệ thống thực sự được sử dụng.

Một thứ tự ưu tiên đáng suy ngẫm: chất lượng trước, chi phí sau

Ở những giai đoạn đầu, trọng tâm hoàn toàn dồn vào việc đạt được độ chính xác và hiệu năng mong muốn cho các chức năng cốt lõi, ngay cả khi điều đó đồng nghĩa với chi phí cao hơn. Lý do là tối ưu chi phí quá sớm có nguy cơ làm suy yếu hiệu quả của hệ thống và cản trở việc người dùng chấp nhận sử dụng nó. Chỉ sau khi đạt được mức độ chính xác và hiệu năng mong muốn, đội ngũ mới bắt đầu tập trung tối ưu chi phí và đảm bảo việc tối ưu đó không ảnh hưởng tiêu cực đến trải nghiệm người dùng hay chất lượng kết quả.

Đây là một bài học đáng để bất kỳ đội ngũ nào xây dựng sản phẩm AI ghi nhớ: sự tin tưởng và thói quen sử dụng của người dùng được xây dựng trước, việc tối ưu vận hành đến sau. Cố gắng làm ngược lại tối ưu chi phí trước khi chứng minh được giá trị thường dẫn đến một sản phẩm rẻ nhưng không ai muốn dùng.

Phản hồi người dùng, dữ liệu giám sát liên tục, và những góc nhìn từ các nhà khoa học chuyên môn liên tục được đưa trở lại vào chu trình phát triển, dẫn đến những cải tiến trong kiến trúc, kỹ thuật truy xuất, hành vi của các agent, và giao diện người dùng nhằm nâng cao hiệu năng, tính khả dụng, và cuối cùng là tác động thực sự đến công việc khoa học.

Chương 15 Kết luận: bài học cho những ai đang xây hệ thống agentic AI của riêng mình

Câu chuyện PRINCE là một minh chứng rằng việc xây dựng một ứng dụng LLM sẵn sàng cho môi trường doanh nghiệp phức tạp như nghiên cứu tiền lâm sàng trong ngành dược là một hành trình đầy thách thức kỹ thuật, nhưng hoàn toàn khả thi. Bằng cách kết hợp một hạ tầng dữ liệu vững chắc, các kỹ thuật truy xuất thông tin tinh vi như RAG và Text-to-SQL, và một hệ thống điều phối đa tác nhân thông minh, ta có thể khai mở những giá trị ẩn giấu trong các kho dữ liệu khổng lồ vốn trước đây gần như không thể tiếp cận được.

Nếu phải rút ra những bài học cốt lõi nhất cho bất kỳ ai đang hoặc sắp xây dựng một hệ thống agentic AI tương tự, đó sẽ là:

- Đừng nhồi mọi thứ vào một prompt. Hãy thiết kế để mỗi bước xử lý chỉ nhận đúng phần ngữ cảnh nó cần đó là kỷ luật ngữ cảnh (context discipline), không phải giới hạn kỹ thuật.
- Xây giàn giáo (harness) mạnh mẽ xung quanh mô hình: lưu trạng thái, thử lại, dự phòng, và các điểm dừng cho con người can thiệp đừng phó mặc mọi thứ cho "trí thông minh" của riêng mô hình.
- Tách bạch các loại phản tư: phản tư về quy trình, phản tư về dữ liệu, và phản tư về bản nháp đầu ra đều cần thiết, và mỗi loại bắt lỗi những vấn đề khác nhau.
- Minh bạch và trích dẫn không phải là tính năng phụ chúng là nền tảng của niềm tin, đặc biệt trong các lĩnh vực có quy định chặt chẽ.
- Đánh giá liên tục, cả trên bộ dữ liệu chuẩn lẫn trên lưu lượng thực tế, theo từng tầng của hệ thống giống như kim tự tháp kiểm thử trong phát triển phần mềm truyền thống.
- Ra mắt sớm, học hỏi liên tục. Ưu tiên độ chính xác và giá trị trước, tối ưu chi phí sau.
- Luôn giữ con người ở vòng lặp cho những quyết định có rủi ro cao hoặc độ tin cậy thấp tự động hoá phần chắc chắn, để con người xử lý phần không chắc chắn.

Bài học rộng hơn từ PRINCE là: một hệ thống agentic AI sẵn sàng cho production không chỉ đến từ những mô hình tốt hơn hay những prompt khéo léo hơn. Sự đáng tin cậy đến từ việc kỹ sư hoá cả ngữ cảnh mà mô hình nhìn thấy, lẫn cái giàn giáo mà mô hình hành động bên trong. Kỹ thuật ngữ cảnh giúp đảm bảo mỗi mô hình có đúng thông tin và chỉ đúng thông tin ở đúng giai đoạn của luồng xử lý. Kỹ thuật giàn giáo giúp

đảm bảo luồng xử lý luôn có ranh giới rõ ràng, có thể quan sát, có thể phục hồi, và phù hợp với một môi trường nghiên cứu được quản lý chặt chẽ.

Khi năng lực của các mô hình tiếp tục cải thiện, một số phần của "giàn giáo" hôm nay có thể trở nên mỏng hơn hoặc được chuyển thành năng lực nội tại của chính mô hình. Nhưng trong các hệ thống nghiên cứu doanh nghiệp đặc biệt là những nơi niềm tin, khả năng truy vết, và khả năng rà soát thực sự quan trọng việc kiểm soát tường minh đối với ngữ cảnh, trạng thái luồng xử lý, khả năng phục hồi, phản tư, và xác thực vẫn sẽ luôn cần thiết.

Hy vọng cuốn sách nhỏ này đã mang lại cho bạn một góc nhìn thực tế và có thể áp dụng được, về những gì thực sự cần thiết để xây dựng và đưa vào production một ứng dụng LLM trong một lĩnh vực vừa giàu dữ liệu, vừa chịu sự quản lý chặt chẽ. Chúc bạn thành công trên hành trình xây dựng hệ thống agentic AI đáng tin cậy của riêng mình.

Chương 16 Ví dụ thực tế 2026: agentic AI ngoài ngành dược

Để thấy rõ hơn rằng những nguyên tắc rút ra từ PRINCE không phải là chuyện riêng của một công ty dược phẩm, chương này đi qua một số triển khai agentic AI đáng chú ý khác đang vận hành trong thực tế tính đến giữa năm 2026 cả những câu chuyện thành công lẫn những bài học từ thất bại. Xuyên suốt các ví dụ, bạn sẽ nhận ra những chủ đề quen thuộc: kỷ luật ngữ cảnh, con người ở vòng lặp, và tầm quan trọng sống còn của khả năng kiểm chứng.

Tài chính: JPMorgan Chase và quy mô vận hành công nghiệp

JPMorgan Chase, một trong những ngân hàng lớn nhất thế giới, vận hành hơn 450 trường hợp sử dụng AI khác nhau trong môi trường production hàng ngày, tính đến năm 2026. Thay vì xây một hệ thống "vạn năng" duy nhất, ngân hàng này áp dụng chiến lược mở rộng theo chiều rộng: mỗi bộ phận nghiệp vụ có những agent chuyên biệt riêng, xử lý các tác vụ hẹp và rõ ràng từ rà soát tuân thủ, phân tích tín dụng, đến hỗ trợ giao dịch. Đây là một minh chứng thực tế cho nguyên tắc "kiến trúc đa tác nhân chuyên biệt hơn một mô hình ôm đồm" mà chúng ta đã thảo luận ở chương 3 và chương 7 khi nói về việc phân cấp Agent Nghiên cứu theo từng lĩnh vực.

Tài chính: Morgan Stanley và bài học về phạm vi sử dụng

Morgan Stanley Wealth Management triển khai một trợ lý nội bộ dựa trên mô hình ngôn ngữ lớn cho khoảng 16.000 cố vấn tài chính, cho phép họ đặt câu hỏi bằng ngôn ngữ tự nhiên và nhận câu trả lời có trích dẫn từ kho nội dung nghiên cứu và tư vấn nội bộ khổng lồ của công ty. Điểm đáng chú ý trong lựa chọn thiết kế của Morgan Stanley là phạm vi sử dụng được giới hạn rất rõ ràng: hệ thống chỉ dành cho người dùng nội bộ (cố vấn tài chính), đóng vai trò hỗ trợ tư vấn chứ không tự ra quyết định thay con người, và luôn truy xuất trên một kho nội dung đã được kiểm duyệt kỹ lưỡng không phải toàn bộ internet. Cách thu hẹp phạm vi có chủ đích này phản ánh chính xác bài học ở chương 2 của cuốn sách này: quyền tự chủ của AI nên tỷ lệ thuận với mức độ rủi ro và mức độ tin cậy đã được kiểm chứng.

Bán lẻ và dịch vụ khách hàng: bài học đảo chiều của Klarna

Có lẽ ví dụ đáng suy ngẫm nhất trong chương này đến từ Klarna, công ty dịch vụ tài chính "mua trước trả sau" của Thụy Điển. Ngay từ khi ra mắt, trợ lý AI của Klarna xử lý 2,3 triệu cuộc hội thoại chỉ trong tháng đầu tiên tương đương khối lượng công việc của 700 nhân viên toàn thời gian, và đến cuối năm 2025, con số đó tăng lên tương đương 853 nhân viên, giúp công ty tiết kiệm ước tính 60 triệu đô la Mỹ. Thời gian xử lý trung bình giảm từ 11 phút xuống dưới 2 phút.

Nhưng câu chuyện không dừng lại ở đó. Vào giữa năm 2025, CEO của Klarna công khai thừa nhận rằng việc quá chú trọng cắt giảm chi phí đã ảnh hưởng tiêu cực đến chất lượng trải nghiệm khách hàng ở một số tình huống đặc biệt là những yêu cầu mang tính cảm xúc hoặc phức tạp, đòi hỏi sự thấu cảm và phán đoán linh hoạt mà một agent thuần túy khó đáp ứng tốt. Klarna sau đó chuyển sang mô hình lai: AI xử lý các yêu cầu thường quy, còn con người tiếp quản khi câu hỏi vượt quá một "ngưỡng tin cậy" nhất định.

Bài học từ Klarna không phải là AI đã thất bại, mà là công ty đã điều chỉnh lại phạm vi sử dụng cho hợp lý. Đây chính xác là nguyên tắc "con người ở vòng lặp cho những quyết định có rủi ro cao hoặc độ tin cậy thấp" đã được nhấn mạnh xuyên suốt cuốn sách này và Klarna đã phải học lại bài học đó bằng thực tiễn, sau khi đã triển khai ở quy mô lớn.

Pháp lý: Harvey AI và cái giá của việc thiếu kỷ luật trích dẫn

Trong lĩnh vực pháp lý, nền tảng Harvey AI đã trở thành hạ tầng nền tảng cho nhiều công ty luật lớn tính đến giữa năm 2026, nền tảng này phục vụ khoảng 100.000 luật sư, đạt doanh thu định kỳ hàng năm khoảng 190 triệu đô la Mỹ, và được khoảng 68% các công ty luật và bộ phận pháp lý nội bộ được khảo sát xác nhận đã triển khai các agent dựa trên Harvey trong công việc thực tế hàng ngày không còn ở giai đoạn thử nghiệm.

Tuy nhiên, ngành pháp lý nói chung cũng đang trải qua một "cuộc khủng hoảng ảo giác" đáng để mọi đội ngũ xây dựng hệ thống agentic AI suy ngẫm: tính đến giữa năm 2026, đã ghi nhận hơn 1.500 vụ việc tại toà án Mỹ liên quan đến việc AI bịa đặt trích dẫn án lệ không có thật, dẫn đến hàng trăm ngàn đô la tiền phạt và ít nhất một vụ luật sư bị đình chỉ hành nghề. Một nghiên cứu độc lập của các nhà nghiên cứu tại Stanford và Yale phát hiện rằng ngay cả những công cụ RAG pháp lý được xây dựng chuyên biệt cho ngành không phải các chatbot thông dụng vẫn tạo ra ảo giác ở tỷ lệ 17-34% trong một số tình huống.

Đây là một minh chứng sống động, mang tính cảnh báo, cho toàn bộ triết lý đã được nhấn mạnh ở chương 10 của cuốn sách này: trích dẫn không chính xác hoặc không thể kiểm chứng không phải là một khiếm khuyết nhỏ có thể bỏ qua nó là rủi ro cốt lõi có thể phá huỷ hoàn toàn giá trị của một hệ thống AI trong lĩnh vực có tính rủi ro cao. Không phải ngẫu nhiên mà PRINCE đầu tư kỹ lưỡng vào cơ chế trích dẫn chi tiết đến từng câu (chương 10) và cơ chế phản tư dữ liệu độc lập trước khi cho phép sinh câu trả lời (chương 8) đây chính xác là những cơ chế phòng ngừa loại rủi ro mà ngành pháp lý đang phải trả giá đắt vì thiếu sót.

Sản xuất và chuỗi cung ứng: RAG trên tài liệu kỹ thuật

Trong lĩnh vực sản xuất, các hệ thống RAG doanh nghiệp được kết nối với sổ tay kỹ thuật, thông số thiết bị, hồ sơ bảo trì, và dữ liệu chuỗi cung ứng, cho phép người vận hành đặt câu hỏi bằng ngôn ngữ tự nhiên và nhận câu trả lời có căn cứ về quy trình, khắc phục sự cố, và tính tương thích phụ tùng được trích nguồn từ tài liệu đã được xác minh. Trong lĩnh vực đấu thầu hợp đồng với chính phủ và dịch vụ kỹ thuật, các hệ thống tương tự đã rút ngắn thời gian đánh giá điều kiện tham gia đấu thầu (RFP) từ hơn hai giờ xuống dưới mười phút, trong khi vẫn duy trì yêu cầu tuân thủ quy định một minh hoạ khác cho giá trị của RAG khi được áp dụng đúng vào một bài toán tra cứu tài liệu phi cấu trúc quy mô lớn, giống hệt bài toán gốc mà PRINCE giải quyết cho ngành dược.

Điểm chung xuyên suốt các ví dụ 2026

Nhìn lại toàn bộ các ví dụ trong chương này, có thể rút ra một vài quan sát chung phản ánh đúng tinh thần của cuốn sách:

- Các tổ chức thành công nhất đều thu hẹp phạm vi sử dụng agent một cách có chủ đích (JPMorgan, Morgan Stanley) thay vì cố gắng để AI làm "mọi thứ" ngay từ đầu.
- Ngay cả những triển khai được truyền thông ca ngợi nhiều nhất (Klarna) cũng phải điều chỉnh lại ranh giới tự chủ của AI dựa trên phản hồi thực tế khẳng định lại nguyên tắc phát triển lặp ở chương 14.
- Ngành nào coi nhẹ kỷ luật trích dẫn và kiểm chứng (như một phần của ngành pháp lý) đang phải trả giá đắt bằng tiền phạt, uy tín, và thậm chí là hậu quả pháp lý nghiêm trọng cho khách hàng cũng cố thêm tầm quan trọng của chương 10 và chương 11 trong cuốn sách này.
- RAG trên tài liệu phi cấu trúc quy mô lớn (sản xuất, pháp lý, dược phẩm) tiếp tục là một trong những ứng dụng agentic AI mang lại giá trị đo lường được rõ ràng nhất trong môi trường doanh nghiệp năm 2026.

Những ví dụ này, dù đến từ những ngành rất khác nhau, đều xác nhận lại thông điệp cốt lõi của cuốn sách: thành công của một hệ thống agentic AI không nằm ở việc chọn được mô hình mạnh nhất, mà nằm ở kỷ luật thiết kế xung quanh mô hình đó đúng ngữ cảnh, đúng giàn giáo, đúng ranh giới tự chủ, và luôn có con người ở những điểm quyết định quan trọng.

Chương 17 Checklist thực hành cho đội ngũ của bạn

Chương này tổng hợp lại các bài học xuyên suốt cuốn sách dưới dạng một danh sách kiểm tra thực tế, dành cho những đội ngũ đang cân nhắc hoặc đang trong quá trình xây dựng một hệ thống agentic AI cho môi trường doanh nghiệp. Đây không phải là một công thức cứng nhắc phải áp dụng y hệt, mà là một tập hợp câu hỏi nên tự đặt ra ở từng giai đoạn.

Trước khi bắt đầu: đánh giá nền tảng dữ liệu

- Dữ liệu của bạn đang phân mảnh ở bao nhiêu hệ thống khác nhau? Có kế hoạch hợp nhất hay ít nhất là chuẩn hoá cách truy cập chưa?
- Bạn có phân biệt rõ giữa dữ liệu có cấu trúc và phi cấu trúc chưa? Chiến lược truy xuất cho mỗi loại nên khác nhau.
- Siêu dữ liệu (metadata) của bạn có đáng tin cậy không, hay cần một cơ chế làm sạch và xác thực liên tục?

Khi thiết kế kiến trúc

- Bạn có đang thiết kế để mỗi bước xử lý chỉ nhận đúng ngữ cảnh nó cần, hay đang có xu hướng nhồi mọi thứ vào một prompt lớn?
- Bạn có một bước làm rõ ý định trước khi hệ thống chọn công cụ hoặc nguồn dữ liệu không?
- Bạn có tách riêng "phản tư về quy trình" và "phản tư về dữ liệu" hay đang gộp chung thành một bước mơ hồ?
- Nếu bạn có nhiều công cụ với phạm vi chồng lấn, bạn đã cân nhắc phân cấp chúng theo lĩnh vực chuyên biệt chưa?

Khi xây dựng độ tin cậy

- Người dùng có thể thấy được các bước trung gian mà hệ thống thực hiện không, hay chỉ thấy kết quả cuối cùng như một hộp đen?
- Mỗi khẳng định trong câu trả lời có trích dẫn cụ thể trở về nguồn không? Trích dẫn có đủ chi tiết để kiểm chứng nhanh (số trang, đoạn trích) không?
- Bạn có kế hoạch đánh giá theo nhiều tầng (giống kim tự tháp kiểm thử) hay chỉ đánh giá đầu-cuối?
- Bạn có cả đánh giá trên bộ dữ liệu chuẩn lẫn đánh giá liên tục trên lưu lượng thực tế không?

Khi thiết kế khả năng phục hồi

- Trạng thái của luồng xử lý có được lưu lại theo từng bước, cho phép phục hồi từ điểm gần nhất, hay phải chạy lại từ đầu khi có lỗi?
- Bạn có cơ chế dự phòng (fallback) sang mô hình hoặc nhà cung cấp khác khi mô hình chính gặp sự cố không?
- Bạn đã xác định rõ ranh giới giữa lỗi nên tự động thử lại và lỗi nên dừng lại chờ con người can thiệp chưa?

Khi vận hành lâu dài

- Bạn có ưu tiên đạt độ chính xác và niềm tin của người dùng trước, rồi mới tối ưu chi phí sau, hay đang cố gắng làm cả hai cùng lúc ngay từ đầu?
- Bạn có đưa hệ thống ra thực tế sớm để thu thập phản hồi, hay đang chờ đến khi "hoàn hảo" mới ra mắt?
- Với những tác vụ có rủi ro cao (ví dụ soạn thảo tài liệu quy định), bạn có đảm bảo luôn có một bước rà soát của con người trước khi kết quả được dùng chính thức không?

Không phải mọi mục trong danh sách này đều áp dụng cho mọi dự án quy mô và mức độ rủi ro của hệ thống bạn xây dựng sẽ quyết định mức độ nghiêm ngặt cần thiết. Nhưng càng đi sâu vào những lĩnh vực có quy định chặt chẽ hoặc rủi ro cao, càng nên nghiêm túc cân nhắc từng mục trong danh sách trên.

Chương 18 Câu hỏi thường gặp

Hệ thống này có thể áp dụng cho ngành khác ngoài dược phẩm không?

Hoàn toàn có thể. Bài toán cốt lõi mà PRINCE giải quyết khai thác tri thức bị chôn vùi trong một kho tài liệu phi cấu trúc khổng lồ, kết hợp với dữ liệu có cấu trúc, trong một môi trường đòi hỏi độ chính xác và khả năng kiểm chứng cao xuất hiện ở rất nhiều ngành khác: pháp lý (hợp đồng, án lệ), tài chính (báo cáo, hồ sơ tuân thủ), bảo hiểm (hồ sơ yêu cầu bồi thường), sản xuất (tài liệu kỹ thuật, hồ sơ chất lượng), và khu vực công (văn bản quy phạm, hồ sơ hành chính). Các nguyên tắc thiết kế kỷ luật ngữ cảnh, ba vòng phản tư, trích dẫn chi tiết, giàn giáo phục hồi lỗi đều mang tính tổng quát, không phụ thuộc vào lĩnh vực dược phẩm cụ thể.

Có nhất thiết phải dùng LangGraph không?

Không nhất thiết. LangGraph là lựa chọn cụ thể của đội ngũ PRINCE, nhưng nguyên tắc quan trọng hơn công cụ: bạn cần một cơ chế điều phối có hỗ trợ lưu trạng thái theo từng bước, cho phép rẽ nhánh có điều kiện và vòng lặp, và tích hợp tốt với hệ thống quan sát (observability). Có nhiều framework khác trên thị trường có thể đáp ứng những yêu cầu này ở các mức độ khác nhau; điều quan trọng là hiểu rõ vì sao những đặc tính đó cần thiết, để có thể đánh giá đúng công cụ phù hợp với bối cảnh của riêng bạn.

Một đội ngũ nhỏ, ít tài nguyên, có thể áp dụng những bài học này không?

Có, dù không nhất thiết phải áp dụng đầy đủ mọi thành phần ngay từ đầu. Bài học về phát triển lặp ở chương 14 chính là câu trả lời: PRINCE cũng không được xây dựng hoàn chỉnh trong một lần, mà tiến hoá qua nhiều năm, bắt đầu từ một hệ thống tìm kiếm đơn giản. Một đội ngũ nhỏ có thể bắt đầu với một pipeline RAG cơ bản có trích dẫn (chương 7 và 10), sau đó dần dần bổ sung các vòng phản tư (chương 6, 8, 9) và cơ chế phục hồi lỗi (chương 12) khi hệ thống mở rộng quy mô và độ phức tạp. Điều quan trọng là hiểu được lý do đằng sau mỗi thành phần, để biết khi nào thực sự cần bổ sung nó.

Làm sao biết khi nào hệ thống của tôi đã "đủ tin cậy" để đưa vào sản xuất?

Không có một ngưỡng số học chung cho mọi hệ thống mức độ tin cậy cần thiết phụ thuộc vào mức độ rủi ro của quyết định mà câu trả lời AI sẽ ảnh hưởng đến. Tuy nhiên, một dấu hiệu thực tế đáng tin cậy hơn một con số đơn lẻ là: liệu người dùng có thể tự kiểm chứng câu trả lời một cách nhanh chóng hay không (chương 10), và liệu có cơ chế nào bắt được các trường hợp hệ thống "tự tin nhưng sai" trước khi chúng gây hậu quả hay không (chương 6, 8, 9, 11). Một hệ thống có độ chính xác chưa hoàn hảo nhưng

minh bạch và dễ kiểm chứng thường an toàn hơn để đưa vào sử dụng so với một hệ thống có độ chính xác cao hơn nhưng hoạt động như một hộp đen.

AI có thay thế hoàn toàn vai trò của chuyên gia con người trong quy trình này không?

Không, và đó không phải là mục tiêu của PRINCE. Xuyên suốt cuốn sách, chúng ta đã thấy nguyên tắc "con người ở vòng lặp" xuất hiện lặp đi lặp lại: từ việc người dùng có quyền điều chỉnh gợi ý nguồn dữ liệu (chương 5), đến việc mọi tài liệu quy định do AI hỗ trợ soạn thảo đều cần chuyên gia rà soát và phê duyệt (chương 9), cho đến việc những trường dữ liệu có độ tin cậy thấp luôn được chuyển cho con người xử lý (chương 13). Mục tiêu của hệ thống không phải là loại bỏ con người khỏi quy trình, mà là loại bỏ phần công việc tìm kiếm, tổng hợp thủ công tốn thời gian để con người có thể dành nhiều thời gian hơn cho phần việc đòi hỏi chuyên môn và phán đoán mà chỉ con người mới làm được.

Phụ lục Bảng thuật ngữ

Dưới đây là các thuật ngữ kỹ thuật xuất hiện xuyên suốt cuốn sách, kèm giải thích ngắn gọn bằng tiếng Việt để tiện tra cứu.

Agent (tác nhân)	Một thành phần AI được giao một vai trò cụ thể trong luồng xử lý, có khả năng suy luận và gọi công cụ để hoàn thành nhiệm vụ của mình.
Agentic AI	Hệ thống AI trong đó mô hình ngôn ngữ không chỉ trả lời một lượt, mà tự lập kế hoạch, gọi công cụ, và điều chỉnh hành động qua nhiều bước để đạt mục tiêu.
Context engineering (kỹ thuật ngữ cảnh)	Việc chọn lọc và định tuyến có chủ đích thông tin mà mỗi mô hình nhận được ở từng giai đoạn xử lý, thay vì đưa toàn bộ thông tin sẵn có vào một prompt.
Harness engineering (kỹ thuật giàn giáo)	Việc xây dựng lớp hạ tầng bao quanh mô hình điều phối, ranh giới công cụ, lưu trạng thái, thử lại, dự phòng, phân tử, quan sát nhằm kiểm soát và duy trì độ tin cậy.
RAG (Retrieval-Augmented Generation)	Kỹ thuật truy xuất tăng cường sinh: tìm các đoạn thông tin liên quan từ kho dữ liệu, rồi đưa vào ngữ cảnh để mô hình sinh câu trả lời có căn cứ.
Text-to-SQL	Kỹ thuật chuyển câu hỏi ngôn ngữ tự nhiên thành câu truy vấn SQL để truy vấn trực tiếp dữ liệu có cấu trúc.
Hybrid retriever (truy xuất lai)	Cách kết hợp tìm kiếm theo ngữ nghĩa vector và tìm kiếm theo từ khóa truyền thống trong cùng một lượt truy xuất.
Reranker (bộ xếp hạng lại)	Một mô hình chuyên dụng đánh giá lại độ liên quan giữa các đoạn văn bản đã truy xuất và câu hỏi gốc, nhằm chọn ra tập kết quả chất lượng cao nhất.
Process reflection (phản tư quy trình)	Đánh giá xem luồng xử lý của agent có đang đi đúng hướng và đúng tiến độ hay không.
Data reflection (phản tư dữ liệu)	Đánh giá xem dữ liệu đã thu thập được có đủ và phù hợp để trả lời câu hỏi hay không.
LangGraph	Một framework mã nguồn mở dùng để xây dựng và điều phối luồng xử lý (workflow) cho các hệ thống agentic, có hỗ trợ lưu trạng thái và xử lý lỗi.
Checkpoint	Cơ chế lưu lại trạng thái của luồng xử lý sau mỗi bước, cho phép hệ thống tiếp tục từ điểm dừng gần nhất thay vì chạy lại từ đầu.
Fallback (dự phòng)	Cơ chế tự động chuyển sang một mô hình hoặc nhà cung cấp thay thế khi lựa chọn chính gặp lỗi hoặc không khả dụng.

Faithfulness (độ trung thực)	Chỉ số đo mức độ câu trả lời sinh ra thực sự được hỗ trợ bởi ngữ cảnh đã cung cấp, không bịa đặt thông tin.
Confidence score (điểm tin cậy)	Điểm số thể hiện mức độ chắc chắn của một kết quả trích xuất hoặc dự đoán tự động, dùng để quyết định có cần con người rà soát hay không.

Code4Life®

Nguồn tham khảo và ghi nhận

Cuốn sách này được viết lại, diễn giải và mở rộng dựa trên nội dung của bài viết:

Sarang Sanjay Kulkarni, "Building Reliable Agentic AI Systems: A Case Study in building production-ready agentic AI systems", martinowler.com, 16 tháng 6 năm 2026.

Bài viết gốc mô tả nền tảng PRINCE (Preclinical Information Center), do Bayer AG phối hợp cùng Thoughtworks phát triển, và cũng dựa trên một bài báo khoa học đăng trên tạp chí Frontiers in Artificial Intelligence. Toàn văn bài viết gốc có thể truy cập tại địa chỉ martinowler.com.

Nội dung trong cuốn sách này được diễn giải hoàn toàn bằng lời văn riêng, không sao chép nguyên văn từ nguồn gốc, nhằm mục đích giáo dục và chia sẻ tri thức phi lợi nhuận cho cộng đồng kỹ thuật nói tiếng Việt. Mọi ghi nhận về công trình gốc, cũng như bản quyền liên quan, thuộc về tác giả bài viết gốc và các tổ chức liên quan (Bayer AG và Thoughtworks).

Nếu bạn muốn tìm hiểu sâu hơn về chủ đề này, chúng tôi khuyến khích bạn đọc trực tiếp bài viết gốc bằng tiếng Anh, cũng như bài báo khoa học liên quan trên Frontiers in Artificial Intelligence.

Lời cảm ơn từ người biên soạn

Xin gửi lời cảm ơn đến tác giả Sarang Sanjay Kulkarni và toàn bộ đội ngũ tại Bayer và Thoughtworks đã công khai chia sẻ những bài học kỹ thuật quý giá này với cộng đồng. Trong một lĩnh vực mà nhiều bài học thực chiến thường bị giữ kín vì lý do cạnh tranh, việc công bố chi tiết đến vậy về kiến trúc, các quyết định kỹ thuật, và cả những gì đã không hiệu quả (như bước rà soát SQL bị loại bỏ ở chương 7) là một đóng góp hiếm có và đáng trân trọng cho cộng đồng kỹ thuật toàn cầu.

Cuốn sách tiếng Việt này được biên soạn với hy vọng đưa những bài học đó đến gần hơn với các kỹ sư, nhà nghiên cứu, và người làm sản phẩm tại Việt Nam những người có thể đang đối mặt với những bài toán tương tự trong chính lĩnh vực của mình, dù đó là dược phẩm, tài chính, pháp lý, hay bất kỳ ngành nào khác đang tìm cách khai phá giá trị từ kho dữ liệu của chính mình.